

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
PROGRAMA DE PÓS-GRADUAÇÃO EM COMPUTAÇÃO

PEDRO ANTÔNIO DA SILVA BORGES

**Implementação de Redes Neurais em
Microcontroladores ESP32 S3 para a
Identificação em Tempo Real de Aedes
aegypti**

Dissertação apresentada como requisito parcial
para a obtenção do grau de Mestre em Ciência da
Computação

Orientador: Prof. Dr. José Rodrigo Furlanetto de
Azambuja

Porto Alegre
2025

CIP — CATALOGAÇÃO NA PUBLICAÇÃO

Borges, Pedro Antônio da Silva

Implementação de Redes Neurais em Microcontroladores ESP32 S3 para a Identificação em Tempo Real de *Aedes aegypti* / Pedro Antônio da Silva Borges. – Porto Alegre: PPGC da UFRGS, 2025.

68 f.: il.

Dissertação (mestrado) – Universidade Federal do Rio Grande do Sul. Programa de Pós-Graduação em Computação, Porto Alegre, BR-RS, 2025. Orientador: José Rodrigo Furlanetto de Azambuja.

1. TinyML. 2. Sistemas Embarcados. 3. *Aedes aegypti*. 4. ESP32-S3. 5. Redes Neurais Convolucionais. 6. Computação na Borda. I. Azambuja, José Rodrigo Furlanetto de. II. Título.

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Reitor: Prof^a. Marcia Barbosa

Vice-Reitor: Prof. Pedro de Almeida Costa

Pró-Reitora de Pós-Graduação: Prof^a. Claudia Wasserman

Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspar

Coordenador do PPGC: Prof. Gabriel Luca Nazar

Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

"Sê humilde para evitar o orgulho, mas voa alto para alcançar a sabedoria."

Santo Agostinho

AGRADECIMENTOS

Agradeço primeiramente a Deus, por permitir eu estar nesta vida sendo instrumento de sua obra. A minha esposa Ana Carolina, por ser minha rocha fundamental, por ser a mão que sustenta quando me faltam forças. Ao meu filhote João Guilherme - o papai abre o caminho para ti seguir a caminhada filhote. Aos meus pais Pedro e Zilda, por vocês eu estou em pé. Aos meus irmãos, que nossa peleja sempre seja abençoada, e que ajoelhar seja sempre para pegar impulso! A Isa, Júnior, José, Naty e Ricardo, por entenderem a ausência e estarem sempre comigo. Seu Verno, por acreditar firmemente em meu potencial, e a Ana Capelão (in Memoriam), Terezinha Borges(in Memoriam) e ao Vô Kindah (in Memoriam) por estarem vibrando aí no céu. Ao meu orientador José Rodrigo Furlanetto Azambuja, por estar sempre presente, mesmo nas maiores tempestades do conhecimento, e a Carlos Francisco Soares de Souza, obrigado por todo o apoio, amizade, e mostrarem a direção.

Obrigado por fazerem parte da minha história.

RESUMO

As doenças transmitidas por vetores, notadamente as arboviroses como Dengue, Zika e Chikungunya, representam um desafio persistente de saúde pública global. No Brasil, o controle do vetor *Aedes aegypti* ainda depende majoritariamente de métodos de vigilância manual, que são onerosos e sofrem de alta latência entre a coleta e a tomada de decisão. Este trabalho propõe uma solução baseada em Internet das Coisas (IoT) e Inteligência Artificial na Borda (*TinyML*) para automatizar a identificação do vetor através da sua assinatura acústica de voo. O sistema foi implementado no microcontrolador ESP32-S3, explorando suas instruções vetoriais para processamento de sinais em tempo real. A metodologia envolveu a construção de um *dataset* híbrido (HumBugDB e Kaggle) e o treinamento de Redes Neurais Convolucionais (CNNs) com diferentes níveis de complexidade. Para viabilizar a execução no hardware restrito, aplicaram-se técnicas de otimização como a extração de características MFCC e a quantização pós-treinamento para inteiros de 8 bits (Int8). Os resultados experimentais demonstraram que a arquitetura de 7 camadas com quantização Int8 atingiu a melhor relação entre desempenho e custo, obtendo uma acurácia de 93,5% e um tempo de inferência de apenas 32,40 ms no *hardware* alvo. Conclui-se que a solução proposta supera as limitações de processamento local, viabilizando armadilhas inteligentes de baixo custo e alta autonomia energética para o monitoramento entomológico em larga escala.

Palavras-chave: TinyML. Sistemas Embarcados. *Aedes aegypti*. ESP32-S3. Redes Neurais Convolucionais. Computação na Borda.

Implementation of Neural Networks on ESP32-S3 Microcontrollers for Real-Time Identification of *Aedes aegypti*

ABSTRACT

Vector-borne diseases, particularly arboviruses such as Dengue, Zika, and Chikungunya, represent a persistent global public health challenge. In Brazil, the control of the *Aedes aegypti* vector still relies heavily on manual surveillance methods, which are costly and suffer from high latency between data collection and decision-making. This work proposes a solution based on the Internet of Things (IoT) and Edge Artificial Intelligence (*TinyML*) to automate vector identification through its acoustic flight signature. The system was implemented on the ESP32-S3 microcontroller, exploiting its vector instructions for real-time signal processing. The methodology involved constructing a hybrid dataset (HumBugDB and Kaggle) and training Convolutional Neural Networks (CNNs) with varying levels of complexity. To enable execution on restricted *hardware*, optimization techniques such as MFCC feature extraction and post-training quantization to 8-bit integers (Int8) were applied. Experimental results showed that the 7-layer architecture with Int8 quantization achieved the best trade-off between performance and cost, reaching an accuracy of 93.5% and an inference time of only 32.40 ms on the target hardware. It is concluded that the proposed solution overcomes local processing limitations, enabling low-cost, high-energy-autonomy smart traps for large-scale entomological monitoring.

Keywords: TinyML, Embedded Systems, *Aedes aegypti*, ESP32-S3, Convolutional Neural Networks, Edge Computing.

LISTA DE FIGURAS

1.1	A arquitetura de computação de borda concentra-se principalmente em recursos de computação distribuídos pela rede e inclui três componentes principais: dispositivos de borda, nós de borda e a nuvem central ou centro de dados.	13
3.1	Representação visual do processo de transformação: o sinal de áudio bruto no domínio do tempo (acima) é convertido em um espectrograma espectro-temporal (abaixo), evidenciando a frequência fundamental e os harmônicos do bater de asas.	36
4.1	Comparação Principal: 3 Camadas (Float32 vs Int8).....	53
4.2	Variante de Precisão: 3 Camadas (Float16 e Int16).....	54
4.3	Comparação Principal: 5 Camadas (Float32 vs Int8).....	55
4.4	Variante de Precisão: 5 Camadas (Float16 e Int16).....	56
4.5	Comparação Principal: 7 Camadas (Float32 vs Int8).....	57
4.6	Variante de Precisão: 7 Camadas (Float16 e Int16).....	58

LISTA DE TABELAS

3.1	Matriz de Experimentos: Arquiteturas vs. Precisão	34
4.1	Comparativo de Desempenho das Arquiteturas no Ambiente de Treinamento <i>Host</i>	51
4.2	Performance de Inferência e Consumo de Recursos no ESP32-S3	60

LISTA DE ABREVIATURAS E SIGLAS

ACEs	Agentes de Combate a Endemias
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DCT	Discrete Cosine Transform
DMA	Direct Memory Access
DSP	Digital Signal Processing
ESP-IDF	Espressif IoT Development Framework
FFT	Fast Fourier Transform
FPU	Floating Point Unit
I2S	Inter-IC Sound
IoT	Internet of Things
MFCC	Mel-Frequency Cepstral Coefficients
ML	Machine Learning
PSRAM	Pseudo Static Random Access Memory
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RMS	Root Mean Square
SoC	System on Chip
SRAM	Static Random Access Memory
STFT	Short-Time Fourier Transform
TinyML	Tiny Machine Learning

SUMÁRIO

1 Introdução	12
2 Fundamentação teórica	17
2.1 A Premissa Biológica: A Batida de Asas como uma Biometria Acústica	17
2.2 A Transformação do Dado: Do Som Bruto à Imagem Espectro-Temporal....	18
2.2.1 Da Limitação Estática à Análise Dinâmica	19
2.2.2 O Algoritmo Cooley-Tukey: Eficiência Computacional	20
2.2.3 A Segmentação Temporal (STFT)	20
2.2.4 Refinamento do Sinal: Janelamento e Preenchimento.....	21
2.3 O Motor de Inteligência: Aprendizado Profundo e Convoluções	22
2.4 O Estado da Arte na Borda e Trabalhos Relacionados	23
2.4.1 Validação Algorítmica	23
2.4.2 Validação de Plataforma (<i>hardware</i>)	24
2.4.3 A Lacuna de Pesquisa	24
2.5 O Ambiente de Execução: A Síntese no <i>Hardware</i> de Borda.....	24
Síntese do Capítulo	25
3 Metodologia de Pesquisa	27
3.1 Delineamento e Materiais	27
3.1.1 <i>Hardware e Firmware</i>	27
3.1.1.1 Ambiente de Treinamento <i>Host</i>	27
3.1.2 Ambiente de Software e Bibliotecas.....	28
3.1.2.1 <i>TensorFlow e Keras</i>	28
3.1.2.2 <i>Librosa</i>	28
3.1.2.3 Ambiente de Execução <i>Target</i>	29
3.1.3 Ambiente de Execução <i>Target</i>	29
3.2 Construção do <i>Dataset</i> e Pré-processamento	30
3.2.1 Matriz Experimental e Definição de Cenários	31
3.2.1.1 Variação de Topologia (Profundidade)	32
3.2.1.2 Variação de Precisão Numérica (Quantização)	32
3.2.2 Implementação da Conversão e Calibração	33
3.3 Padronização de Sinais e Eficiência Espectral.....	34
3.4 Engenharia de Características: A Escolha pelos MFCCs.....	35
3.5 Balanceamento via Data Augmentation Assimétrico	37
3.5.1 Estratégia para a Classe Alvo (<i>Aedes aegypti</i>)	37
3.5.2 Estratégia para a Classe de Controle (Não- <i>Aedes</i>)	38
3.6 Implementação e Fundamentação Algorítmica	38
3.7 Pré-processamento e Extração de Características.....	39
3.8 Arquitetura da Rede Neural Convolucional.....	40
3.9 Desenvolvimento do Sistema Embarcado	43
3.10 Implementação do <i>Firmware</i> e Processamento em Tempo Real	43
3.10.1 Aquisição de Dados via Interface I2S e DMA.....	43
3.10.2 Cadeia de Pré-processamento de Sinal (DSP)	44
3.10.2.1 Filtragem e Remoção de Componente DC.....	44
3.10.2.2 Janelamento (Windowing).....	45
3.10.2.3 Transformada Rápida de Fourier (FFT)	46
3.10.2.4 Extração de MFCCs e DCT.....	46

3.10.3	Motor de Inferência TinyML	47
3.10.4	Lógica de Decisão e Eficiência Energética	49
4	Resultados e Discussão	50
4.1	Análise de Desempenho Computacional no Ambiente de Treinamento	50
4.1.1	Discussão dos Dados de Treinamento	50
4.1.2	Arquitetura de 3 Camadas (Leve)	52
4.1.3	Arquitetura de 5 Camadas (Intermediária)	55
4.1.4	Arquitetura de 7 Camadas (Robusta)	57
4.2	Análise da Implementação Embarcada e Eficiência	57
4.2.1	Otimização de Processamento de Sinais (DSP)	58
4.2.2	Análise Crítica: O Desempenho da Arquitetura 7 Camadas (Int8) ...	59
4.2.2.1	O Paradoxo da Precisão: Quantização como Regulari- zador	59
4.2.2.2	Eficiência de <i>Hardware</i> e Latência	60
4.2.3	Desempenho Computacional no ESP32-S3	60
4.2.4	Discussão dos Resultados Embarcados	61
5	Conclusão	62
5.1	Síntese dos Resultados	62
5.2	Contribuições da Pesquisa	63
5.3	Limitações e Trabalhos Futuros	63
5.4	Limitações e Trabalhos Futuros	64
	Referências	66

1 INTRODUÇÃO

Baseado na prova de conceito inicial demonstrada por Borges, Dal Castel e Azambuja (BORGES; AZAMBUJA; CASTEL, 2025), e na análise detalhada de implementação e performance realizada por Dal Castel (CASTEL, 2023), esta dissertação avança a pesquisa investigando o próximo passo crítico para a viabilidade prática: a otimização do modelo. Enquanto os trabalhos anteriores validaram a portabilidade e o desempenho base, este trabalho foca na aplicação de técnicas como a quantização de modelo e a introdução de arquiteturas de pré-filtragem para analisar os *trade-offs* entre acurácia, latência e consumo energético, visando uma solução final ainda mais eficiente e adequada para uma implantação em larga escala.

As doenças transmitidas por vetores, notadamente pelo mosquito *Aedes aegypti*, representam um dos maiores e mais persistentes desafios de saúde pública no Brasil e em diversas outras nações tropicais. Longe de ser um problema controlado, o *Aedes aegypti* continua a ser o vetor de arboviroses de grande impacto, como a Dengue, Zika e Chikungunya. A situação da Dengue, em particular, atingiu níveis alarmantes. Até meados de 2025, o Brasil já enfrenta um dos piores surtos epidêmicos de sua história, registrando milhões de casos prováveis e milhares de óbitos confirmados. Estes números não apenas representam uma tragédia humanitária, mas também impõem uma pressão esmagadora sobre os sistemas de saúde e geram um fardo econômico substancial, estimado em bilhões de reais anuais devido a custos com tratamento e perda de produtividade (Painel de Monitoramento de Arboviroses do Ministerio da Saude, 2025).

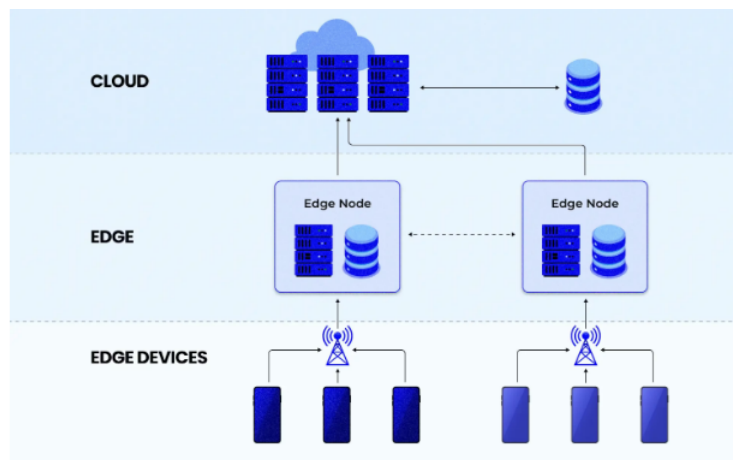
A estratégia predominante para o controle do vetor ainda se baseia em métodos reativos, como o uso de inseticidas, e em um sistema de vigilância entomológica largamente manual. Este sistema depende de armadilhas de captura (*ovitrapms*) que são inspecionadas por agentes de saúde em ciclos de visitação periódicos. Tal abordagem, embora fundamental, sofre de severas limitações operacionais: é intensiva em mão de obra, dispendiosa e, crucialmente, lenta. O tempo decorrido entre a captura de um mosquito em uma armadilha, sua coleta, identificação em laboratório e a subsequente ação de controle pode levar dias ou semanas. Nesta janela de tempo, um novo foco do mosquito pode já ter se estabelecido, tornando as ações de bloqueio menos eficazes e permitindo que a transmissão da doença ocorra (Vigilancia Entomologica e Controle Vetorial do Ministerio da Saude, 2025).

Historicamente, as soluções de IoT baseavam-se em uma arquitetura centralizada,

onde dispositivos periféricos atuavam apenas como coletores de dados, transmitindo informações brutas para processamento em servidores remotos ou na nuvem *Cloud Computing*. Embora esse modelo ofereça capacidade computacional praticamente ilimitada, ele impõe desafios críticos para aplicações de monitoramento ambiental em tempo real: a alta latência na transmissão, a dependência de conectividade estável e, crucialmente, o consumo excessivo de largura de banda necessário para transmitir fluxos contínuos de dados complexos, como áudio.

Neste contexto, surge o paradigma da Computação de Borda *Edge Computing*. Diferente da abordagem centralizada, a computação de borda propõe a descentralização do poder de processamento, movendo a análise dos dados para a periferia da rede, fisicamente próxima à fonte de geração da informação. Essa mudança de arquitetura reduz drasticamente o tráfego de dados, diminui o tempo de resposta e aumenta a privacidade e a segurança da aplicação.

Figura 1.1: A arquitetura de computação de borda concentra-se principalmente em recursos de computação distribuídos pela rede e inclui três componentes principais: dispositivos de borda, nós de borda e a nuvem central ou centro de dados.



Fonte: (PANARIN, 2024)

Contudo, uma confluência de avanços tecnológicos recentes abre uma nova e promissora fronteira para a modernização desta vigilância. A ascensão da Internet das Coisas (IoT) viabilizou a criação de redes de sensores de baixo custo, capazes de coletar dados do ambiente e transmiti-los em tempo real. Paralelamente, os avanços em Inteligência Artificial na Borda *Edge AI*, especificamente no campo emergente do *Tiny Machine Learning* (TinyML), tornaram possível executar modelos de aprendizado de máquina diretamente em microcontroladores de baixíssimo consumo energético e custo. Esta revolução permite que a "inteligência" que antes residia na nuvem seja deslocada para a ponta, onde os dados são gerados (WARDEN; SITUNAYAKE, 2020).

É neste cenário, caracterizado pela convergência entre desafios críticos de saúde pública e novas oportunidades tecnológicas, que esta dissertação se insere. Situado na fronteira da Engenharia de Sistemas Embarcados aplicada à Epidemiologia Digital, o presente trabalho propõe o desenvolvimento integral, a implementação e a avaliação de desempenho de um sistema de vigilância entomológica inteligente, operando de forma autônoma e projetado sob rigorosos critérios de baixo custo para viabilizar sua escalabilidade.

A arquitetura da solução consiste em um dispositivo de borda *edge device* ativado por som, estruturado como uma armadilha eletrônica não invasiva. O sistema integra sensores acústicos de alta fidelidade, configurados para capturar o espectro de frequências audíveis do ambiente, com foco na assinatura sonora específica gerada pelo batimento de asas dos mosquitos. O núcleo de processamento é constituído por um microcontrolador da família ESP32-S3, escolhido por sua capacidade de processamento e suporte a instruções vetoriais, essenciais para a execução de algoritmos de Inteligência Artificial.

Para a identificação da espécie, o *firmware* embarca um modelo classificador baseado em Redes Neurais Convolucionais (CNN). Este modelo foi treinado, quantizado e otimizado especificamente para operar em *hardware* com restrições severas de memória e energia, permitindo a distinção precisa entre o *Aedes aegypti* e outras fontes sonoras ou espécies não vetoras.

Ao realizar a inferência e a classificação do inseto *in loco* e em tempo real, o sistema proposto visa eliminar a latência inerente aos processos manuais de vistoria, que muitas vezes apresentam uma defasagem de semanas entre a coleta e a consolidação dos dados. Essa agilidade permite uma mudança de paradigma na gestão sanitária: as autoridades de saúde podem transitar de uma postura reativa para uma atuação proativa. Com a detecção imediata da presença do vetor, torna-se possível otimizar a alocação de recursos humanos e insumos, direcionando os Agentes de Combate a Endemias (ACEs) para focos ativos com precisão cirúrgica, potencializando a eficácia das ações de controle e prevenindo a eclosão de surtos epidêmicos.

É neste cenário de desafio de saúde pública e oportunidade tecnológica que esta dissertação se insere. O presente trabalho propõe o desenvolvimento e a avaliação de um sistema de vigilância entomológica inteligente, autônomo e de baixo custo. A solução consiste em uma armadilha equipada com um sensor acústico capaz de capturar o som característico da batida de asas dos mosquitos e um classificador baseado em Redes Neurais Convolucionais (CNN), embarcado em um microcontrolador da família ESP32. Ao

realizar a classificação do mosquito in loco e em tempo quase real, o sistema visa eliminar a latência do processo manual e permitir que as autoridades de saúde respondam à detecção de um vetor perigoso de forma imediata, otimizando a alocação de recursos e potencializando a eficácia das ações de controle.

Para guiar esta pesquisa, a seguinte questão central é proposta: "É viável desenvolver um classificador de mosquitos *Aedes aegypti* que opere com boa acurácia e baixo tempo de inferência em um microcontrolador de baixíssimo custo, utilizando o som da batida das asas como dado de entrada, e quais são os *trade-offs* de performance ao aplicar técnicas de otimização como a quantização de modelos?"

Para responder a esta questão, os seguintes objetivos específicos foram traçados:

Desenvolver e treinar um modelo de Rede Neural Convolucional para a classificação acústica do *Aedes aegypti*.

Implementar (portar) o modelo treinado para um microcontrolador ESP32, analisando o impacto da precisão numérica (double vs. single-precision) nos resultados e no tempo de processamento.

Aplicar técnicas de quantização (int8) ao modelo, avaliando sistematicamente os *trade-offs* entre acurácia, uso de memória, e latência da inferência.

Propor e avaliar uma arquitetura de sistema em cascata, com um algoritmo de pré-filtragem, como estratégia para otimização do consumo energético e do tempo de processamento.

Analisar os resultados de forma comparativa, fornecendo um panorama claro sobre a factibilidade e as limitações da aplicação de TinyML para vigilância de vetores em cenários com restrição de recursos.

A presente dissertação está organizada em cinco capítulos, estruturados de forma a guiar o leitor desde os conceitos fundamentais até a validação experimental do sistema proposto.

O Capítulo 2 (Fundamentação Teórica) estabelece o alicerce multidisciplinar da pesquisa. Inicia-se com a premissa biológica da identificação acústica do vetor, transita pelas técnicas de Processamento Digital de Sinais (transformadas de Fourier e espectrogramas) necessárias para a conversão dos dados, fundamenta o uso de Redes Neurais Convolucionais para reconhecimento de padrões e contextualiza o estado da arte da computação na borda *Edge Computing*.

O Capítulo 3 (Metodologia de Pesquisa) detalha o rigor científico e de engenharia adotado. Descreve-se a construção do *dataset* híbrido e as estratégias de *Data Augmen-*

tation para balanceamento de classes. A seção define a matriz experimental, combinando variações de topologia de rede e precisão numérica, e documenta a implementação do *firmware* no microcontrolador ESP32-S3, com ênfase nas otimizações de DSP, aquisição via DMA (Acesso Direto à Memória), e inferência via TinyML.

O Capítulo 4 (Resultados e Discussão) consolida a análise dos dados em duas frentes: a avaliação da convergência e desempenho computacional dos modelos no ambiente de treinamento *Host* e a validação crítica da implementação embarcada *Target*. Discutem-se a eficiência das otimizações de sinal, o impacto da quantização na latência e o consumo de recursos no *hardware* final.

Por fim, o Capítulo 5 (Conclusão) retoma a questão de pesquisa à luz dos resultados obtidos, sintetiza as contribuições técnicas do trabalho, discute suas implicações, aponta as limitações do estudo e sugere direções para trabalhos futuros visando a evolução da solução.

2 FUNDAMENTAÇÃO TEÓRICA

O presente capítulo tem por finalidade estabelecer o arcabouço teórico e conceitual que sustenta o desenvolvimento desta pesquisa. Dada a natureza multidisciplinar do problema abordado, optou-se por não realizar apenas uma revisão bibliográfica fragmentada, mas sim construir uma narrativa integradora que une domínios do conhecimento historicamente distintos. Esta base é fundamental para compreender como fenômenos biológicos podem ser traduzidos em dados digitais e, subsequentemente, interpretados por algoritmos computacionais em ambientes com restrição de recursos.

A estruturação deste capítulo segue a lógica do fluxo de dados no sistema proposto. Inicialmente, a discussão parte da biologia vetorial e da epidemiologia, focando especificamente no comportamento acústico do *Aedes aegypti*. Compreender a assinatura espectral do voo do mosquito e as nuances do dimorfismo sexual da espécie não é apenas uma curiosidade biológica, mas o pré-requisito essencial para a correta definição dos parâmetros de engenharia.

Na sequência, o texto transita para o Processamento Digital de Sinais (PDS), detalhando as técnicas matemáticas necessárias para converter ondas sonoras brutas em representações tempo-frequência (espectrogramas) inteligíveis para modelos computacionais. Esta etapa serve de ponte para o domínio do Aprendizado de Máquina e Redes Neurais Convolucionais (CNNs), onde são explorados os mecanismos que permitem a extração automática de características e a classificação de padrões complexos.

Por fim, a fundamentação converge para a Engenharia de Sistemas Embarcados e o paradigma do TinyML. É neste ponto que se justificam as escolhas de *hardware* e *software* — especificamente o uso do microcontrolador ESP32-S3 — demonstrando como é possível portar modelos robustos de Inteligência Artificial, tradicionalmente executados em servidores de alta performance, para a "borda"(edge), viabilizando assim uma vigilância entomológica contínua, autônoma e escalável.

2.1 A Premissa Biológica: A Batida de Asas como uma Biometria Acústica

O ponto de partida para o desenvolvimento de qualquer sistema de classificação automatizada reside na identificação de uma característica que seja, simultaneamente, distintiva entre as classes de interesse e mensurável por sensores eletrônicos. No contexto da vigilância entomológica, a literatura científica oferece uma base biológica robusta para a

utilização do espectro sonoro como vetor de identificação. O trabalho seminal de (GIBSON; RUSSELL, 2006) é fundamental para esta validação, pois transcende a observação passiva ao demonstrar que a frequência da batida de asas não é um subproduto acidental da locomoção, mas sim um componente crítico e evolutivamente refinado do ritual de acasalamento e reconhecimento interespecífico.

Os autores elucidam que a interação acústica, especificamente a convergência harmônica entre machos e fêmeas, atua como um mecanismo de isolamento reprodutivo, garantindo que o acasalamento ocorra apenas entre indivíduos da mesma espécie. Esta especificidade eleva a assinatura acústica de um mero "ruído" ambiental para o patamar de uma biometria comportamental e morfológica. A frequência fundamental do voo é ditada por restrições físicas invariáveis, como a massa corporal do inseto e a área de superfície de suas asas, o que confere a este sinal uma estabilidade temporal e espectral que poucos outros bioindicadores possuem.

Dessa forma, a hipótese central deste trabalho ganha plausibilidade: se os próprios mosquitos desenvolveram, ao longo de milênios de pressão evolutiva, sistemas sensoriais (antenas) sintonizados para distinguir essas frequências em ambientes ruidosos, infere-se que um sistema computacional, devidamente treinado, possa emular essa capacidade. Portanto, a escolha de um sensor acústico (microfone) como elemento transdutor não é arbitrária; ela se ancora em um mecanismo de seleção natural consolidado, fornecendo uma justificativa sólida e bioinspirada para toda a cadeia de processamento digital e inteligência artificial que será detalhada nesta dissertação.

2.2 A Transformação do Dado: Do Som Bruto à Imagem Espectro-Temporal

Tendo estabelecido a premissa biológica de que o bater de asas constitui uma impressão digital acústica, o desafio de engenharia subsequente consiste em converter o sinal bruto capturado pelo microfone em um formato inteligível para algoritmos de aprendizado de máquina. O dado nativo, proveniente do Conversor Analógico-Digital (ADC), apresenta-se como uma série temporal unidimensional de variações de pressão (voltagem) no domínio do tempo. Embora esta representação contenha toda a informação física do evento, ela é intrinsecamente ruidosa, de alta dimensionalidade e computacionalmente ineficiente para a extração direta de padrões complexos.

A simples análise da amplitude do sinal ao longo do tempo não é capaz de distinguir, por exemplo, o zumbido harmônico de um mosquito do ruído estocástico de vento

ou tráfego urbano. Portanto, faz-se necessária uma mudança de paradigma: transmutar o problema de Processamento Digital de Sinais (DSP) em um problema de Visão Computacional.

Este processo constitui uma *pipeline* crítica de pré-processamento, cuja função é converter a onda sonora 1D em uma representação bidimensional densa — a imagem espectro-temporal. Nesta etapa, decisões matemáticas rigorosas sobre janelamento, resolução de frequência e escalas psicoacústicas definem a qualidade da informação que será entregue à Inteligência Artificial. O objetivo final é gerar uma "topografia acústica" onde as características discriminantes do vetor (frequência fundamental e harmônicos) sejam realçadas como texturas visuais, tornando-as "computacionalmente tratáveis" e permitindo que a Rede Neural Convolucional identifique padrões com robustez, independentemente das variações de amplitude ou fase do sinal original.

2.2.1 Da Limitação Estática à Análise Dinâmica

A abordagem clássica para análise de frequência reside na Transformada Rápida de Fourier (FFT), cujo algoritmo seminal proposto por (COOLEY; TUKEY, 1965) revolucionou o processamento digital ao reduzir drasticamente a complexidade computacional do cálculo espectral. No entanto, a aplicação direta da FFT impõe uma limitação fundamental para o propósito deste trabalho: ela opera sob a premissa de que o sinal é estacionário. Em termos estatísticos, isso implica assumir que as propriedades de frequência do sinal são constantes e invariáveis durante todo o intervalo de observação.

Essa suposição colide frontalmente com a natureza do problema. Aplicar uma FFT sobre uma gravação longa de áudio resulta em um espectro médio global, que nos informa com precisão *quais* frequências ocorreram, mas omite completamente a informação de *quando* ocorreram. Para um sinal dinâmico, intermitente e transiente como o voo de um inseto — que pode se aproximar, afastar ou pousar em questão de milissegundos —, essa visão estática é insuficiente. Ao integrar a energia do sinal ao longo de todo o vetor de tempo, a FFT colapsa a dimensão temporal, fazendo com que um breve zumbido de mosquito se dilua ou se misture irremediavelmente com ruídos de fundo contínuos, impossibilitando a distinção entre um evento biológico complexo e uma interferência estática.

2.2.2 O Algoritmo Cooley-Tukey: Eficiência Computacional

A viabilidade de executar análises espectrais em tempo real no microcontrolador ESP32-S3 deve-se fundamentalmente à eficiência algorítmica. Enquanto a definição clássica da Transformada Discreta de Fourier (DFT) para um sinal $x[n]$ de comprimento N exige uma complexidade computacional de $O(N^2)$, o algoritmo introduzido por (COOLEY; TUKEY, 1965) reduz esse esforço para $O(N \log_2 N)$.

O princípio central do método, conhecido como "dividir para conquistar", decompõe a DFT de N pontos em duas DFTs menores de $N/2$ pontos: uma contendo as amostras de índices pares e outra as de índices ímpares. A reconstrução do espectro total é dada pela equação:

$$X[k] = E[k] + W_N^k \cdot O[k] \quad (2.1)$$

onde:

- $E[k]$ é a DFT dos componentes pares ($x[0], x[2], \dots$);
- $O[k]$ é a DFT dos componentes ímpares ($x[1], x[3], \dots$);
- $W_N^k = e^{-j\frac{2\pi}{N}k}$ é o fator de fase complexo, denominado *twiddle factor*.

Essa decomposição recursiva forma a estrutura conhecida como "Diagrama Borboleta". Para um sistema embarcado, essa otimização é crítica: para uma janela de análise típica de 1024 amostras, o algoritmo de (COOLEY; TUKEY, 1965) é aproximadamente 100 vezes mais rápido que a implementação direta da fórmula da DFT, permitindo que o processamento do áudio do mosquito ocorra dentro das restrições de tempo de um ciclo de leitura do sensor.

2.2.3 A Segmentação Temporal (STFT)

Para superar a supressão da dimensão temporal inerente à análise global, adota-se a Transformada de Fourier de Curto Período (STFT), conforme formalizado por (PORTNOFF, 1980). A STFT resolve o problema da não-estacionariedade ao aplicar o princípio da quase-estacionariedade local: assume-se que, embora o sinal mude ao longo de segundos, suas propriedades espectrais permanecem razoavelmente constantes dentro de janelas muito curtas (na ordem de milissegundos).

Operacionalmente, o algoritmo segmenta o sinal contínuo em janelas temporais deslizantes e parcialmente sobrepostas (*overlapping*), aplicando a Transformada de Fourier individualmente em cada fragmento. Essa abordagem permite mapear a evolução espectral do sinal ao longo do tempo, gerando uma matriz bidimensional onde é possível observar flutuações de frequência instantâneas. No entanto, esta técnica impõe um compromisso fundamental regido pelo Princípio da Incerteza de Heisenberg-Gabor: janelas temporais muito curtas oferecem excelente localização no tempo (sabemos quando o mosquito passou), mas pobre resolução de frequência; inversamente, janelas longas melhoram a discriminação tonal, mas borram a precisão temporal. O ajuste deste parâmetro é, portanto, vital para capturar a assinatura acústica do vetor com fidelidade.

2.2.4 Refinamento do Sinal: Janelamento e Preenchimento

A segmentação do sinal, embora mandatória para a análise temporal, introduz artefatos matemáticos decorrentes da natureza finita da amostragem. Ao cortar abruptamente um trecho de áudio, impõe-se implicitamente uma "Janela Retangular" ao sinal. Isso cria descontinuidades severas nas bordas do vetor de dados que, ao serem processadas pela FFT, manifestam-se como o fenômeno de "vazamento espectral" *spectral leakage*. Neste cenário, a energia que deveria estar concentrada estritamente na frequência fundamental do inseto "vaza" para frequências vizinhas através de lóbulos laterais (*side lobes*) de alta amplitude, borrando a análise e mascarando sinais mais fracos.

Para mitigar este efeito deletério, aplica-se a técnica de janelamento *windowing*, discutida extensivamente por (HARRIS, 1978). O uso de funções de janela não-retangulares (como *Hann* ou *Hamming*) atua multiplicando o sinal por uma curva em forma de sino, suavizando as bordas do segmento para zero antes do cálculo da transformada. Essa operação reduz drasticamente as descontinuidades de borda e suprime os lóbulos laterais, garantindo que o lóbulo principal da frequência seja destacado em detrimento do ruído de fundo.

Complementarmente, a técnica de preenchimento com zeros *zero-padding* detalhada por (DONNELLE; RUST, 2005), é empregada para refinar a apresentação dos dados. Ao estender o vetor temporal com uma sequência de zeros antes da FFT, não se aumenta a resolução espectral intrínseca (que depende da duração da janela), mas sim a densidade de pontos do espectro resultante. Isso atua como uma interpolação trigonométrica ideal, permitindo uma visualização muito mais suave e detalhada das características

espectrais finas, facilitando a identificação precisa do pico de frequência fundamental do bater de asas, mesmo que este caia entre dois bins da FFT original.

subsectionA Síntese Visual: O Espectrograma O produto matemático imediato da STFT é uma matriz de números complexos, onde cada célula $X(t, f)$ contém informações tanto de magnitude quanto de fase. No entanto, para fins de classificação por aprendizado de máquina em bioacústica, a informação de fase — que dita o alinhamento exato das ondas senoidais — é frequentemente descartada em favor da energia do sinal. Ao calcularmos a magnitude ao quadrado desses valores complexos ($|X(t, f)|^2$), geramos o Espectrograma de Potência (ALMEIDA, 1994). Esta operação de redução efetivamente transmuta a natureza do problema: o que era originalmente uma série temporal unidimensional torna-se uma imagem bidimensional, onde o eixo das abcissas representa o tempo, o eixo das ordenadas a frequência e a "cor" da *pixel* a intensidade (dB). Esta transformação é o passo mais crucial da engenharia de características, pois nos permite reformular um problema de Processamento Digital de Sinais (DSP) como um problema clássico de Visão Computacional. Ao tratar o som como uma imagem, "abrem-se as portas" para o uso do ferramental mais robusto disponível na atualidade para reconhecimento de padrões: as Redes Neurais Convolucionais (CNNs). Diferente de algoritmos tradicionais que analisam estatísticas globais, as CNNs são excepcionalmente hábeis em explorar a correlação local entre *pixels* vizinhos, permitindo identificar texturas e formas visuais complexas — neste caso, a assinatura harmônica vertical e a continuidade temporal horizontal que compõem a "forma" única do voo do *Aedes aegypti*.

2.3 O Motor de Inteligência: Aprendizado Profundo e Convoluções

Uma vez que o sinal acústico foi transmutado para o domínio da frequência, gerando a representação visual do espectrograma, o desafio de engenharia desloca-se do processamento de sinais para o reconhecimento de padrões. Para esta tarefa, o paradigma do Aprendizado Profundo (*Deep Learning*) consolidou-se como a solução predominante. Conforme fundamentado por (LECUN; BENGIO; HINTON, 2015), a capacidade destas redes de aprender representações de dados com múltiplos níveis de abstração permitiu avanços disruptivos em tarefas de percepção computacional.

Dentro deste vasto campo, as Redes Neurais Convolucionais (CNNs) emergem como a arquitetura do estado da arte para lidar com dados estruturados em grade, como é o caso das imagens geradas pelos espectrogramas. Diferente das redes neurais tradicionais

(totalmente conectadas), que falham em capturar dependências espaciais locais, as CNNs utilizam filtros (kernels) que percorrem a entrada para extrair características hierárquicas — desde bordas simples até formas complexas.

A aplicação de CNNs ultrapassa o reconhecimento de objetos em fotografias convencionais. Estudos recentes demonstram sua eficácia robusta em cenários variados, desde a detecção de pistas em navegação autônoma (CHAO; YU-PEI; YA-JIE, 2019) até o reconhecimento de padrões biológicos na pecuária (QI; BAIYANG; LAN, 2021). No contexto acústico, a CNN interpreta o espectrograma tratando o eixo temporal e o eixo de frequência como dimensões espaciais. A propriedade de “invariância à translação” das convoluções é particularmente valiosa aqui: ela permite que a rede identifique a assinatura harmônica do mosquito (a “forma” do som) independentemente de em qual momento da janela de áudio ela ocorre, garantindo robustez contra variações temporais na captura do sinal.

2.4 O Estado da Arte na Borda e Trabalhos Relacionados

A aplicação de técnicas de Aprendizado Profundo especificamente para a vigilância entomológica tem sido objeto de investigação recente, estabelecendo os precedentes sobre os quais este trabalho se alicerça. A literatura atual pode ser categorizada em dois momentos distintos: a validação algorítmica (o “quê”) e a validação de plataforma (o “onde”).

2.4.1 Validação Algorítmica

No âmbito da classificação taxonômica via áudio, o trabalho de (FERNANDES; CORDEIRO; RECAMONDE-MENDOZA, 2021) apresenta-se como o marco de referência primário. Os autores demonstraram a viabilidade técnica de utilizar Redes Neurais Convolucionais para distinguir, com alta acurácia, o *Aedes aegypti* de outras espécies e ruídos ambientais. A contribuição crucial deste estudo foi provar que o espectrograma do voo contém entropia suficiente para ser separável por uma CNN. Contudo, as experimentações foram conduzidas em ambientes computacionais de alto desempenho (*desktops/servidores*), sem as restrições severas de energia e memória inerentes a um dispositivo de campo autônomo.

2.4.2 Validação de Plataforma (*hardware*)

O passo seguinte em direção à aplicabilidade real — a transição para a computação de borda — foi explorado na dissertação de (CASTEL, 2023). Este trabalho focou nos desafios de engenharia de software e hardware necessários para portar o classificador inteligente para um microcontrolador da família ESP32-S3. (CASTEL, 2023) estabeleceu um *baseline* de funcionamento, provando que o *hardware* possui as instruções vetoriais e a capacidade de memória mínima necessária para executar a inferência neural proposta.

2.4.3 A Lacuna de Pesquisa

Embora o estado da arte demonstre que o modelo é eficaz ((FERNANDES; CORDEIRO; RECAMONDE-MENDOZA, 2021)) e sua execução na borda é tecnicamente possível ((CASTEL, 2023)), resta uma lacuna crítica relacionada à otimização extrema. A literatura ainda carece de uma análise comparativa profunda sobre os limites da quantização (redução da precisão numérica de *float32* para *int8*) e da simplificação arquitetural (redução de camadas). Não está claro até que ponto é possível comprimir o modelo para ganhar eficiência energética e velocidade de inferência sem sacrificar a confiabilidade da detecção abaixo de um limiar operacional seguro. É nesta fronteira de eficiência do TinyML que esta pesquisa atua, buscando o ponto ótimo entre o custo computacional e a precisão biológica.

2.5 O Ambiente de Execução: A Síntese no *Hardware* de Borda

A etapa final desta construção teórica consiste em ancorar o modelo de inteligência abstrato ao mundo físico. A execução remota em servidores de nuvem (*Cloud Computing*), embora ofereça poder computacional ilimitado, mostra-se impraticável para uma rede de armadilhas distribuídas geograficamente em áreas urbanas ou remotas. Fatores como a instabilidade da conectividade, a latência de rede e, sobretudo, o custo energético e financeiro da transmissão contínua de dados de áudio inviabilizam a abordagem centralizada.

A solução, portanto, emerge do paradigma da Computação na Borda (*Edge Computing*), conforme discutido por (SELIMI et al., 2020), (NETO; FONSECA; GAMA,

2020) e (GIANG et al., 2018). Esta arquitetura preza pelo processamento local do dado, movendo a decisão para o ponto de coleta.

A materialização deste conceito ocorre através de Sistemas Embarcados (HENKEL et al., 2012), notadamente utilizando microcontroladores de baixo custo e alta eficiência energética, como a família ESP32 (ESPRESSIF, 2022). O desafio de engenharia reside no fato de que este *hardware* impõe restrições severas de memória (SRAM) e capacidade de processamento (clock). A viabilidade de executar uma Rede Neural Convolucional (CNN) — tipicamente demandante de recursos — em um dispositivo tão limitado só é possível através de um ecossistema de software altamente especializado. Ferramentas como o *TensorFlow Lite for Microcontrollers* (GOOGLE, 2022) são essenciais neste processo, pois permitem a quantização e a otimização dos pesos da rede, traduzindo modelos complexos para uma linguagem que o microcontrolador possa interpretar eficientemente.

Síntese do Capítulo

Em suma, a fundamentação teórica exposta ao longo deste capítulo consolidou o arcabouço científico necessário para transformar um fenômeno natural em uma solução de engenharia. A revisão da literatura confirma que a hipótese desta pesquisa se sustenta sobre quatro pilares interdependentes:

1. **A Base Biológica:** A validação de que a assinatura acústica do voo não é um artefato aleatório, mas um traço fenotípico evolutivamente estável e distinto, regido pelo dimorfismo sexual e pela biomecânica do inseto (GIBSON; RUSSELL, 2006). Isso garante que o "dado" de entrada possui qualidade e discriminabilidade intrínsecas suficientes para ser classificado.
2. **O Processamento de Sinais:** A confirmação de que algoritmos clássicos, notadamente a Transformada de Fourier de Curto Período (STFT) aliada ao janelamento, constituem o método matemático robusto e computacionalmente viável para traduzir a variação de pressão sonora temporal em representações espectrais ricas em informação (COOLEY; TUKEY, 1965; HARRIS, 1978).
3. **A Inteligência Artificial:** O reconhecimento de que as Redes Neurais Convolucionais (CNNs) representam o estado da arte para a extração automática de características em dados espectrais. A literatura demonstra que a topologia das CNNs é ideal para capturar as invariâncias de tempo e frequência inerentes ao som do

mosquito, superando métodos tradicionais de *Machine Learning* (FERNANDES; CORDEIRO; RECAMONDE-MENDOZA, 2021).

4. **A Viabilidade Técnica:** A evidência empírica de que a execução deste fluxo complexo em dispositivos de borda *Edge Devices* já transborda o campo teórico. Trabalhos anteriores de prova de conceito atestam que microcontroladores modernos possuem o conjunto de instruções necessário para realizar tais inferências (CASTEL, 2023).

Contudo, a análise crítica do estado da arte revela que a viabilidade técnica, por si só, não é sinônimo de exequibilidade operacional para uma implantação em larga escala. A literatura cobriu extensivamente a validação do "conceito"(funcionalidade), mas deixou em aberto a validação da "eficiência"(otimização).

Questões cruciais permanecem sem resposta definitiva: qual é o limite de compressão (quantização) que o modelo suporta antes de perder sua acurácia biológica? Como equilibrar o trade-off fino entre a profundidade da rede neural e a autonomia energética de um dispositivo que deve operar por semanas com bateria? É precisamente sobre esta lacuna de otimização entre o algoritmo ideal e o *hardware* real que a metodologia deste trabalho se debruçará a seguir.

3 METODOLOGIA DE PESQUISA

Este capítulo detalha os procedimentos metodológicos, materiais e técnicas analíticas empregadas para o desenvolvimento, otimização e validação do sistema embarcado de classificação de mosquitos. A abordagem adota um delineamento experimental e quantitativo, visando isolar variáveis de arquitetura de rede neural e precisão numérica para avaliar o desempenho em *hardware* de borda.

3.1 Delineamento e Materiais

A pesquisa foi conduzida em um ambiente controlado híbrido, segregado em duas etapas computacionais distintas: o treinamento dos modelos (realizado em estação de trabalho de alto desempenho – *Host*) e a validação de inferência (realizada no sistema embarcado alvo – *Target*).

3.1.1 *Hardware e Firmware*

Para garantir a reprodutibilidade dos experimentos, os ambientes de *hardware* e software foram definidos conforme as especificações a seguir.

3.1.1.1 *Ambiente de Treinamento Host*

Todo o pipeline de pré-processamento de áudio, extração de espectrogramas e treinamento das Redes Neurais Convolucionais (CNNs) foi executado em um computador desktop configurado para computação científica. A utilização de uma GPU dedicada foi mandatória para viabilizar o treinamento das múltiplas arquiteturas propostas em tempo hábil. As especificações técnicas da estação de trabalho são:

- **Processador (CPU):** Intel Core (TM)i3-2330M @ 2.20GHz
- **Memória RAM:** 8,00 GB DDR3
- **Placa de Vídeo (GPU):** Intel(R) HD Graphics 3000(32MB)
- **Armazenamento:** SSD 112 GB
- **Sistema Operacional:** Windows 10 Enterprise

3.1.2 Ambiente de Software e Bibliotecas

A implementação dos algoritmos de inteligência artificial e processamento digital de sinais foi realizada sobre a linguagem de programação *Python*, versão 3.11.2. A escolha desta versão específica deve-se às melhorias significativas de desempenho no interpretador *CPython* introduzidas na série 3.11, oferecendo uma execução mais célere das rotinas de tratamento de dados em comparação às versões anteriores (3.9 ou 3.10). O ecossistema de desenvolvimento foi estruturado em torno de duas bibliotecas fundamentais:

3.1.2.1 *TensorFlow e Keras*

Para a modelagem, treinamento e avaliação das Redes Neurais Convolucionais, utilizou-se o *framework TensorFlow* na versão 2.19.0 (GOOGLE, 2022). Esta ferramenta foi selecionada não apenas pela sua API de alto nível *Keras*, que facilita a prototipagem rápida de arquiteturas complexas, mas principalmente pela sua suíte de ferramentas voltadas para dispositivos embarcados.

O papel do *TensorFlow* nesta metodologia estende-se além do treinamento; ele é responsável pela etapa crítica de conversão do modelo. Utilizou-se o *TFLite Converter* para transformar os modelos treinados em ponto flutuante (*float32*) para o formato *FlatBuffer* (*.tflite*), permitindo a aplicação subsequente de técnicas de quantização pós-treinamento necessárias para a execução no microcontrolador ESP32-S3.

3.1.2.2 *Librosa*

O pré-processamento acústico foi conduzido através da biblioteca *Librosa* (MC-FEE et al., 2015), que representa o padrão da indústria para análise de áudio e música em *Python*. A biblioteca abstrai a complexidade matemática das operações de DSP (*Digital Signal Processing*), garantindo a precisão numérica na conversão dos sinais.

Neste trabalho, a *Librosa* foi empregada para duas funções primárias:

1. **Normalização e Reamostragem:** Garantir que todos os áudios do *dataset*, independentemente da fonte original, fossem convertidos para uma taxa de amostragem consistente (ex: 16 kHz ou 44.1 kHz) antes do processamento.
2. **Geração de Espectrogramas:** A aplicação prática da Transformada de Fourier de Curto Período (STFT), conforme fundamentado no capítulo anterior. A biblioteca

gerenciou a aplicação das janelas de Hann e a conversão das escalas de amplitude para decibéis (escala logarítmica), gerando as matrizes visuais que alimentam as CNNs.

3.1.2.3 Ambiente de Execução Target

O dispositivo de borda selecionado para a validação experimental foi o microcontrolador ESP32-S3 (ESPRESSIF SYSTEMS, 2023). Esta escolha representa uma atualização estratégica em relação às gerações anteriores (como a série ESP32-WROVER), fundamentada na arquitetura do seu processador Xtensa® Dual-Core 32-bit LX7.

3.1.3 Ambiente de Execução Target

O dispositivo de borda selecionado para a validação experimental foi o microcontrolador ESP32-S3 (ESPRESSIF SYSTEMS, 2023). Esta escolha não representa apenas uma atualização incremental em relação às gerações anteriores (como a série ESP32-WROVER), mas sim uma decisão estratégica fundamentada na arquitetura do seu processador Xtensa® Dual-Core 32-bit LX7.

A principal motivação para a adoção do ESP32-S3, é em no seu suporte nativo residir as instruções vetoriais (*vector instructions*), projetadas especificamente para acelerar cargas de trabalho de Inteligência Artificial e Processamento Digital de Sinais (DSP). Essas extensões permitem a execução de operações SIMD (*Single Instruction, Multiple Data*), possibilitando que o processador realize múltiplas operações de multiplicação e acumulação (MAC) em paralelo num único ciclo de *clock*. Para Redes Neurais Convolucionais, que dependem massivamente de operações matriciais, essa capacidade traduz-se em uma redução drástica na latência de inferência e uma eficiência energética superior, pois a CPU passa menos tempo ativa para processar o mesmo volume de dados.

Adicionalmente, a hierarquia de memória do módulo foi determinante. O dispositivo conta com 512 KB de SRAM interna de alta velocidade e suporte a PSRAM Octal (*Pseudo Static RAM*) externa. Estes recursos são críticos para a viabilidade do projeto: a SRAM interna acomoda os *tensors* de ativação e os *buffers* de áudio em tempo real, garantindo acesso imediato, enquanto a PSRAM oferece a elasticidade necessária para carregar os pesos de arquiteturas de CNN mais profundas (como a variante de 7 camadas) sem incorrer em erros de estouro de memória (*Out of Memory* - OOM).

Para a captação de áudio, rejeitou-se o uso de microfones analógicos convencionais em favor da especificação de microfones digitais MEMS com interface I2S, especificamente o modelo INMP441 (INVENSENSE, 2024). A escolha pela interface I2S (*Inter-IC Sound*) é estratégica: ao realizar a conversão analógico-digital diretamente no encapsulamento do sensor, o sistema torna-se imune a ruídos eletromagnéticos que poderiam ser induzidos nos cabos durante a transmissão do sinal até o microcontrolador. Este componente garante uma alta relação sinal-ruído (SNR) e uma resposta de frequência plana na faixa de interesse (400 Hz – 2 kHz), sendo capaz de capturar com fidelidade as nuances espectrais da batida de asas do *Aedes aegypti* mesmo em ambientes não controlados.

O desenvolvimento do *firmware* foi realizado sobre o framework oficial ESP-IDF v5.0. Esta versão do ambiente de desenvolvimento oferece um diferencial crítico: o suporte nativo e otimizado a operações de DSP (*Digital Signal Processing*) através da biblioteca `esp-dsp`, que implementa transformadas rápidas de Fourier (FFT) em *assembly*. Além disso, o framework provê integração profunda com o interpretador *TensorFlow Lite for Microcontrollers* (WARDEN; SITUNAYAKE, 2020). Neste projeto, utilizou-se a biblioteca `ESP-NN`, que substitui os *kernels* de referência genéricos do *TensorFlow* por implementações otimizadas que fazem uso explícito das instruções vetoriais do processador S3, maximizando o *throughput* da inferência neural..

3.2 Construção do *Dataset* e Pré-processamento

A premissa central de modelos de Deep Learning estabelece que a capacidade de generalização da rede é estritamente limitada pela qualidade e representatividade dos dados de treinamento. No contexto de TinyML aplicado à vigilância entomológica, enfrenta-se um desafio duplo: a escassez de dados rotulados de alta qualidade e a necessidade de o modelo operar em ambientes ruidosos não controlados.

Para endereçar estes desafios, esta pesquisa rejeitou o uso de uma fonte única de dados. Optou-se estrategicamente pela construção de um *dataset* híbrido, consolidando amostras de duas origens distintas para maximizar a heterogeneidade acústica:

HumBugDB (KISKIN et al., 2021): Utilizado como a fonte primária de verdade taxonômica *Ground Truth*. Este repositório acadêmico fornece gravações de *Aedes aegypti* capturadas sob protocolos rigorosos. Sua inclusão garante que o modelo aprenda as características bioacústicas fundamentais e sutis do inseto.

Repositório Kaggle (KAGGLE, 2017): Utilizado como fonte de variância estocástica. As amostras provenientes desta fonte, muitas vezes capturadas por dispositivos móveis ou em condições subótimas, introduzem uma diversidade de perfis de ruído de fundo e características de microfones *device-specific noise*.

A fusão destas bases, resultando em um corpus inicial de 1.487 arquivos, é uma decisão metodológica para prevenir o viés de domínio. Se o treinamento fosse restrito a dados de laboratório (limpos), o modelo sofreria degradação severa ao ser implantado em uma armadilha real na rua. A hibridização força a rede neural a discriminar o padrão do inseto independentemente do nível de ruído ou do *hardware* de gravação.

Para isso, a função abaixo demonstra a sobre robustez e variabilidade dos dados

```

1 # Funcoes para aumentar a variabilidade do dataset
2 def add_noise(data, noise_factor=0.005):
3     noise = np.random.randn(len(data))
4     return data + noise_factor * noise
5
6 def change_volume(data, gain_min=0.3, gain_max=1.0):
7     gain = np.random.uniform(gain_min, gain_max)
8     return data * gain
9
10 def shift_time(data, shift_max=0.2):
11     shift = np.random.randint(int(SAMPLE_RATE * shift_max))
12     return np.roll(data, shift)
13
14 def change_pitch(data, sample_rate, n_steps=2.0):
15     steps = np.random.uniform(-n_steps, n_steps)
16     return librosa.effects.pitch_shift(data, sr=sample_rate, n_steps=
    steps)

```

Listing 3.1: Funções de Aumento de Dados (Data Augmentation)

3.2.1 Matriz Experimental e Definição de Cenários

Para determinar o ponto de operação ótimo entre acurácia biológica e eficiência computacional, não se optou por uma arquitetura única. Em vez disso, estabeleceu-se uma matriz experimental fatorial composta por 12 cenários distintos, resultantes da combinação de três topologias de rede neural (variável de complexidade) e quatro formatos de precisão numérica (variável de otimização).

O *script* de treinamento desenvolvido em *Python* atua como um orquestrador parametrizável, capaz de instanciar, treinar e converter cada uma das combinações descritas a seguir.

3.2.1.1 Variação de Topologia (Profundidade)

A arquitetura base segue o padrão VGG (*Visual Geometry Group*), caracterizada pelo empilhamento sequencial de blocos de convolução seguidos por redução de dimensionalidade (*MaxPooling*). As três variantes avaliadas foram desenhadas para testar os limites de capacidade de abstração versus o consumo de memória SRAM:

- **Arquitetura Leve (3 Camadas):** Composta por apenas um bloco de extração de características (2 camadas convolucionais) seguido pelo classificador denso. Projetada para ser o limite inferior de latência e consumo, testando se características visuais simples são suficientes para distinguir o espectrograma.
- **Arquitetura Balanceada (5 Camadas):** Adiciona um segundo bloco de convolução, totalizando 4 camadas de extração + 1 densa. Representa o compromisso teórico entre profundidade e desempenho.
- **Arquitetura Robusta (7 Camadas):** A versão mais profunda avaliada (apresentada no código de referência), contendo três blocos convolucionais completos. Esta topologia visa maximizar a extração de características não-lineares complexas, testando o limite superior de processamento do ESP32-S3.

3.2.1.2 Variação de Precisão Numérica (Quantização)

Cada uma das três arquiteturas foi submetida a quatro estratégias de representação numérica durante a etapa de conversão via `TFLiteConverter`. Esta etapa é crítica, pois define como os pesos e ativações são armazenados e processados pelas instruções vetoriais do processador Xtensa LX7:

1. **Float32 (Baseline):** O formato padrão de ponto flutuante de precisão simples (32 bits). Serve como a "verdade fundamental" de acurácia, mas impõe o maior custo de memória e processamento, não usufruindo de aceleração de *hardware* específica.
2. **Float16 (Half-Precision):** Reduz o tamanho de armazenamento do modelo pela metade (16 bits), mantendo a dinâmica de ponto flutuante. Avalia-se se a perda de precisão decimal afeta a discriminação dos harmônicos sutis do mosquito.

3. **Int16 (Quantização de Alta Precisão):** Converte os pesos para inteiros de 16 bits. Este formato é um meio-termo, oferecendo uma faixa dinâmica superior ao Int8, ideal para sinais de áudio onde a amplitude varia significativamente, mas com menor compressão.
4. **Int8 (Quantização Total):** O cenário de maior otimização. Pesos e ativações são comprimidos para inteiros de 8 bits (1 byte). Este formato é o alvo primário das instruções vetoriais do ESP32-S3, prometendo a maior aceleração de inferência e a menor ocupação de memória (4x menor que Float32), sob o risco de degradação da acurácia devido a erros de arredondamento.

3.2.2 Implementação da Conversão e Calibração

Para viabilizar as variantes quantizadas (`int8` e `int16`), o *pipeline* de software implementou uma rotina de calibração utilizando um *Representative Dataset*. Conforme observado no código, um gerador (`representative_data_gen`) alimenta o conversor com 500 amostras reais do conjunto de treinamento durante o processo de compilação do modelo.

```

1      # Gerador para calibrar a faixa dinamica dos inteiros (Int8)
2      def representative_data_gen():
3          # Usa 500 amostras reais do treino para calibracao
4          for input_value in tf.data.Dataset.from_tensor_slices(X_train.
astype(np.float32)).batch(1).take(500):
5              yield [input_value]
6
7      # Configuracao da Conversao
8      converter = tf.lite.TFLiteConverter.from_keras_model(model)
9      converter.optimizations = [tf.lite.Optimize.DEFAULT]
10     converter.representative_dataset = representative_data_gen
11
12     # Forca operacoes puramente inteiras (compativel com ESP32-S3
Vector Inst.)
13     converter.target_spec.supported_ops = [tf.lite.OpsSet.
TFLITE_BUILTINS_INT8]
14     converter.inference_input_type = tf.int8
15     converter.inference_output_type = tf.int8
16

```

17

```
tflite_model = converter.convert()
```

Listing 3.2: Configuração do Conversor TFLite e Dataset Representativo

Esta etapa é indispensável: ela permite que o interpretador *TensorFlow Lite* analise estatisticamente a distribuição dos valores de ativação (min/max) e calcule os fatores de escala e ponto zero (*zero-point*) necessários para mapear os números reais para inteiros discretos sem destruir a informação espectral do som do mosquito.

A Tabela 3.1 sumariza os modelos gerados e avaliados neste estudo.

Tabela 3.1: Matriz de Experimentos: Arquiteturas vs. Precisão

Precisão / Profundidade	3 Camadas (Leve)	5 Camadas (Médio)	7 Camadas (Robusto)
Float32	Modelo 3L-F32	Modelo 5L-F32	Modelo 7L-F32
Float16	Modelo 3L-F16	Modelo 5L-F16	Modelo 7L-F16
Int16	Modelo 3L-I16	Modelo 5L-I16	Modelo 7L-I16
Int8	Modelo 3L-I8	Modelo 5L-I8	Modelo 7L-I8

3.3 Padronização de Sinais e Eficiência Espectral

A heterogeneidade das fontes exigiu uma etapa rigorosa de normalização. Todos os arquivos de áudio foram convertidos para canal único (mono) e reamostrados *down-sampling* para uma taxa de 8 kHz. Esta decisão baseia-se em dois pilares argumentativos: Fundamentação Biológica e Teórica: Conforme o Teorema de Nyquist-Shannon (ELSEVIER, 2025), uma taxa de amostragem de 8 kHz permite a reconstrução perfeita de frequências até 4 kHz. A literatura entomológica (GIBSON; RUSSELL, 2006) demonstra que a frequência fundamental do bater de asas do *Aedes aegypti* reside na faixa de 400 Hz a 600 Hz, com seus primeiros harmônicos raramente excedendo 2 kHz. Portanto, 8 kHz cobre o espectro de interesse com ampla margem de segurança. Otimização de Hardware: Taxas de amostragem superiores (como 44.1 kHz) gerariam vetores de dados cinco vezes maiores, sem adicionar informação relevante para a classificação da espécie. No contexto do ESP32-S3, reduzir a dimensionalidade na entrada resulta em menor consumo de memória RAM para os buffers de áudio e menor custo computacional nas etapas subsequentes de pré-processamento.

3.4 Engenharia de Características: A Escolha pelos MFCCs

Em aplicações de áudio com Deep Learning, raramente utiliza-se o sinal bruto no domínio do tempo, devido à sua alta dimensionalidade e variância de fase. A escolha recaiu sobre os Coeficientes Cepstrais de Frequência Mel (MFCCs), em detrimento de espectrogramas lineares simples. A justificativa para o uso de MFCCs (DAVIS; MERMELSTEIN, 1980) é tripla: Percepção Não-Linear: A aplicação da escala Mel redistribui as bandas de frequência para dar maior resolução às baixas frequências (onde está o som do mosquito) e menor às altas, filtrando ruídos irrelevantes de alta frequência. Descorrelação: A etapa de Transformada Discreta de Cosseno (DCT) no cálculo dos MFCCs descorrelaciona os coeficientes. Isso é particularmente benéfico para Redes Neurais, pois facilita a convergência do treinamento ao apresentar features mais ortogonais. Compacidade: Enquanto um espectrograma convencional pode gerar uma matriz densa e grande, os MFCCs condensam a informação espectral em poucos coeficientes (neste trabalho, 13), reduzindo drasticamente o tamanho do tensor de entrada da CNN para (40×13) , viabilizando inferências rápidas.

O *script* abaixo, mostra a configuração dos parâmetros de áudio 8 kHz e a extração dos MFCCs.

```

1 SAMPLE_RATE = 8000          # Downsampling para economizar memoria
2 N_FFT = 512                 # Janela de analise
3 HOP_LENGTH = 256           # Sobreposicao
4 SAMPLES_PER_FILE = 40      # Tamanho fixo da entrada temporal
5
6 def extract_features(audio):
7     try:
8         # Garante tamanho minimo para FFT
9         if len(audio) < N_FFT:
10             audio = np.pad(audio, (0, N_FFT - len(audio)))
11
12         # Gera Espectrograma Mel
13         S = librosa.feature.melspectrogram(
14             y=audio, sr=SAMPLE_RATE, n_fft=N_FFT,
15             hop_length=HOP_LENGTH, n_mels=32
16         )
17         S_db = librosa.power_to_db(S, ref=1.0)
18
19         # Extrai MFCCs (diminuindo dimensionalidade)

```

```

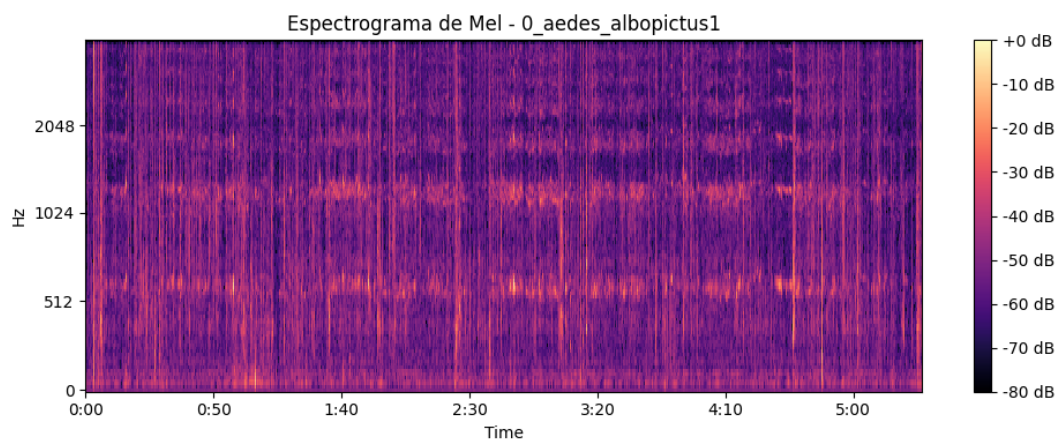
20 mfcc = librosa.feature.mfcc(S=S_dB, n_mfcc=13)
21
22 # Ajuste de tamanho fixo (Padding ou Corte)
23 if mfcc.shape[1] > SAMPLES_PER_FILE:
24     mfcc = mfcc[:, :SAMPLES_PER_FILE]
25 else:
26     pad_width = SAMPLES_PER_FILE - mfcc.shape[1]
27     mfcc = np.pad(mfcc, pad_width=((0, 0), (0, pad_width)),
mode='constant')
28
29     return mfcc.T # Transposta para formato (Tempo, Features)
30 except Exception as e:
31     return None

```

Listing 3.3: Rotina de Extração de MFCCs e Padronização

Para visualizar a natureza dos dados processados pela rede neural, a Figura 3.1 apresenta um exemplo real de um Espectrograma Mel gerado a partir de uma das amostras do *dataset*. Nesta representação, o eixo horizontal corresponde ao tempo, o eixo vertical representa as bandas de frequência ajustadas à escala Mel, e a escala de cores (mapa de calor) indica a intensidade sonora em decibéis. Observa-se como o ruído de fundo e o sinal do inseto são distribuídos espectralmente, formando a 'imagem' que será interpretada pelos kernels da CNN.

Figura 3.1: Representação visual do processo de transformação: o sinal de áudio bruto no domínio do tempo (acima) é convertido em um espectrograma espectro-temporal (abaixo), evidenciando a frequência fundamental e os harmônicos do bater de asas.



Fonte: Autoria própria (2025).

3.5 Balanceamento via Data Augmentation Assimétrico

A análise preliminar do *dataset* consolidado revelou uma assimetria severa na distribuição das classes: o corpus continha 1.067 amostras da classe negativa ("Não-Aedes" ou Classe 0) contra apenas 420 amostras da classe positiva ("Aedes" ou Classe 1).

No contexto de aprendizado supervisionado, treinar uma Rede Neural Convolucional com tal disparidade conduziria inevitavelmente ao "Paradoxo da Acurácia". Neste cenário, a função de perda (*loss function*) convergiria para um mínimo local onde o modelo optaria por classificar todas as entradas como a classe majoritária. Embora isso pudesse resultar em uma acurácia global nominalmente alta (aprox. 72%), a sensibilidade (*recall*) para o vetor alvo seria nula, inutilizando o sistema como ferramenta de vigilância epidemiológica.

Para corrigir este viés estatístico sem descartar informações valiosas da classe majoritária, desenvolveu-se um protocolo de **Aumento de Dados Assimétrico e Seletivo**. Diferente das abordagens tradicionais que aplicam transformações idênticas a todo o conjunto, esta metodologia ajusta a intensidade e a natureza da geração de dados sintéticos inversamente à frequência da classe e diretamente às características físicas do sinal.

3.5.1 Estratégia para a Classe Alvo (*Aedes aegypti*)

Para a Classe 1, foram utilizadas 420 amostras, onde aplicou-se um fator de multiplicação de $5\times$. O objetivo aqui transcende a mera replicação numérica; busca-se a exploração do espaço latente para cobrir a variabilidade biológica e ambiental do vetor. As transformações específicas incluíram:

- **Pitch Shifting (Deslocamento de Tom):** Esta é a transformação de maior valor semântico para o problema. Biologicamente, a frequência fundamental do bater de asas não é uma constante universal; ela varia em função do tamanho corporal do inseto, da temperatura ambiente e da idade do espécime (GIBSON; RUSSELL, 2006). Ao alterar matematicamente o tom das gravações originais (sem alterar a duração), o algoritmo simula a existência de mosquitos com diferentes morfologias, tornando a rede capaz de generalizar para além dos indivíduos específicos capturados no laboratório.

- **Time Shifting (Deslocamento Temporal):** Consiste na rotação cíclica (*rolling*) do vetor de áudio. Como a inferência no ESP32-S3 ocorre em janelas deslizantes contínuas, o som do mosquito pode iniciar em qualquer ponto do *buffer* de entrada. Esta técnica força o modelo a aprender a característica de "invariância à translação temporal", ou seja, reconhecer a assinatura do inseto independentemente de sua posição relativa no início ou fim da janela de amostragem.
- **Simulação de Ganho e Ruído:** Variações de amplitude foram aplicadas para simular a distância física entre o inseto e o microfone (Lei do inverso do quadrado da distância), garantindo que o modelo detecte tanto vetores próximos (sinal forte/saturado) quanto distantes (sinal fraco).

3.5.2 Estratégia para a Classe de Controle (Não-Aedes)

Para a Classe 0, foram utilizadas 1067 amostras, onde aplicou-se um fator conservador de $3\times$. A lógica subjacente é que a classe de "fundo" já possui uma alta variância natural (vozes, carros, vento, outros insetos). Portanto, não é necessário criar novas variantes espectrais complexas, mas sim aumentar a robustez das existentes.

Limitou-se o processamento à **Injeção de Ruído Gaussiano** e **Atenuação de Ganho**. Estas operações ensinam a rede neural que a presença de estática ou a baixa intensidade do sinal não são características exclusivas de uma classe, prevenindo que o modelo associe erroneamente "silêncio" ou "chiado" à presença (ou ausência) do mosquito.

O resultado deste protocolo foi a equalização das densidades de probabilidade das classes, permitindo que o classificador aprenda fronteiras de decisão baseadas na morfologia do sinal espectral, e não na probabilidade a priori de ocorrência dos eventos.

3.6 Implementação e Fundamentação Algorítmica

A construção do classificador baseou-se em uma *pipeline* de processamento de áudio e aprendizado profundo, implementada em linguagem *Python*. A escolha das bibliotecas e funções não foi arbitrária, mas guiada por princípios de Processamento Digital de Sinais (PDS) e otimização de redes neurais. Como o objetivo é a redução de dimensionalidade e a extração de informações relevantes, áudio bruto contém muita redundância; o pré-processamento visa entregar à rede neural apenas a "assinatura" do sinal conforme

detalhado a seguir.

3.7 Pré-processamento e Extração de Características

A etapa de pré-processamento, realizada com auxílio da biblioteca *Librosa* (MC-FEE et al., 2015), tem por objetivo reduzir a dimensionalidade do sinal bruto preservando apenas a assinatura espectral relevante para a discriminação da espécie.

- **Reamostragem e Teorema de Nyquist:** O carregamento dos áudios utilizou a função de reamostragem para 8 kHz, obtido através da função 3.3. Esta decisão fundamenta-se no Teorema da Amostragem de Nyquist-Shannon (SHANNON, 1949), que estabelece que a taxa de amostragem deve ser o dobro da maior frequência de interesse, a taxa escolhida captura a informação essencial com ampla margem, otimizando o uso de memória reduzindo o volume de dados em mais de 5 vezes, o que torna-se importante para sistemas embarcados (ESP32-S3), pois diminui o uso de memória RAM e o tempo de inferência.
- **Escala Mel e Logaritmo:** A conversão do sinal para o domínio da frequência utilizou a Transformada de Fourier de Curto Termo (STFT), seguida pelo mapeamento na Escala Mel. A Escala Mel mimetiza a percepção auditiva não-linear, oferecendo maior resolução em baixas frequências (STEVENS; VOLKMANN; NEWMAN, 1937). Posteriormente, aplicou-se a conversão de amplitude para Decibéis (escala logarítmica), comprimindo a faixa dinâmica do sinal e facilitando a convergência numérica da rede neural, Como a informação crítica do mosquito está nas baixas frequências, a escala Mel atua como um "zoom" na região de interesse, descartando ruídos de alta frequência que poderiam confundir a rede.
- **MFCCs (Mel-Frequency Cepstral Coefficients):** A representação final adotada foi a dos MFCCs. Esta técnica aplica uma Transformada Discreta de Cosseno (DCT) sobre o espectrograma Mel, resultando em coeficientes descorrelacionados que representam o envelope espectral do som (DAVIS; MERMELSTEIN, 1980). Foram extraídos 13 coeficientes, gerando tensores de entrada compactos ideais para processamento na borda.

3.8 Arquitetura da Rede Neural Convolutacional

A arquitetura selecionada fundamenta-se no paradigma das Redes Neurais Convolutacionais (CNNs), reconhecidas como o estado da arte para o reconhecimento de padrões em dados estruturados em grade topológica, como é o caso das imagens espectrais geradas no pré-processamento (LECUN; BENGIO; HINTON, 2015). A escolha por uma CNN, em detrimento de Perceptrons Multicamadas (MLP) ou Redes Recorrentes (RNNs), justifica-se pela sua eficiência em capturar dependências locais e pela capacidade de compartilhamento de pesos, o que reduz drasticamente o número total de parâmetros do modelo — um requisito mandatório para a viabilidade no ESP32-S3.

A estrutura interna das redes avaliadas (nas variações de 3, 5 e 7 camadas) compõe-se do empilhamento sequencial dos seguintes blocos funcionais, cada um desempenhando um papel matemático distinto na extração de conhecimento:

- Camadas Convolutacionais (Conv2D):** Constituem o núcleo de processamento da rede. Estas camadas realizam a extração de características locais através da operação matemática de convolução, onde filtros (*kernels*) treináveis de dimensão 3×3 deslizam sobre o espectrograma de entrada. Diferente de uma camada densa, a *Conv2D* preserva a relação espacial entre os eixos de tempo e frequência. As camadas iniciais aprendem a detectar padrões espectrais de baixo nível, como linhas verticais (ataques sonoros) ou horizontais (frequências contínuas), enquanto as camadas mais profundas combinam essas informações para reconhecer a estrutura harmônica complexa e a assinatura temporal do voo do mosquito.
- Batch Normalization:** Inserida imediatamente após as convoluções e antes da função de ativação, esta técnica normaliza a saída de cada camada para ter média zero e variância unitária. O objetivo primário é reduzir o fenômeno do *Internal Covariate Shift* — a mudança na distribuição das entradas da camada durante o treinamento. Conforme demonstrado por Ioffe e Szegedy (IOFFE; SZEGEDY, 2015), isso não apenas acelera a convergência do treinamento, permitindo taxas de aprendizado maiores, mas atua como um elemento crucial para a futura quantização. Ao manter as ativações dentro de uma faixa dinâmica estável e previsível, a *Batch Normalization* minimiza o erro de discretização quando os valores de ponto flutuante são convertidos para inteiros de 8 bits (*Int8*).
- Ativação ReLU:** A função *Rectified Linear Unit*, definida matematicamente como

$f(x) = \max(0, x)$, foi adotada para introduzir não-linearidade ao modelo. Sem esta não-linearidade, a rede neural comportar-se-ia como um simples regressor linear, incapaz de modelar as fronteiras de decisão complexas necessárias para distinguir espécies de insetos.

Além de mitigar o problema do desvanecimento do gradiente (*vanishing gradient*) típico de funções sigmóides em redes profundas (NAIR; HINTON, 2010), a ReLU possui uma vantagem crítica para sistemas embarcados: sua implementação computacional é extremamente barata (uma simples comparação com zero), economizando ciclos de *clock* preciosos no processador LX7.

- **Max Pooling:** Esta camada realiza a subamostragem (*downsampling*) dos mapas de características, retendo apenas o valor máximo dentro de uma janela deslizante (tipicamente 2×2).

Este processo cumpre duas funções vitais: primeiro, reduz a dimensionalidade espacial dos dados em 75% a cada aplicação, diminuindo o custo computacional das camadas subsequentes; segundo, confere à rede a propriedade de **invariância à translação** (GOODFELLOW; BENGIO; COURVILLE, 2016). Isso significa que o modelo torna-se capaz de detectar a assinatura sonora do *Aedes aegypti* independentemente de sua posição exata no tempo (início ou fim da janela de áudio) ou de pequenas variações na frequência fundamental.

- **Dropout:** Aplicado exclusivamente durante a fase de treinamento nas camadas densas finais, o *Dropout* é uma técnica de regularização estocástica. Ela opera desativando aleatoriamente uma fração dos neurônios (neste trabalho, 50%) a cada iteração do algoritmo de otimização.

Isso impede que os neurônios desenvolvam co-adaptações complexas e frágeis, forçando a rede a aprender características robustas e distribuídas (SRIVASTAVA et al., 2014). O resultado é um modelo com maior capacidade de generalização, prevenindo o *overfitting* (sobreajuste) — um risco alto quando se treina redes profundas em *datasets* de tamanho limitado.

Abaixo podemos ver a implementação da arquitetura utilizada como exemplo, sendo utilizada nas outras CNNs, diminuindo a quantidade das camadas de redes neurais.

```

1     model = tf.keras.Sequential([
2         tf.keras.layers.Input(shape=X_train.shape[1:]),
3
4         # Bloco 1 (2 Camadas Convolucionais)

```

```

5         tf.keras.layers.Conv2D(32, (3, 3), padding='same', use_bias=
False),
6         tf.keras.layers.BatchNormalization(),
7         tf.keras.layers.Activation('relu'),
8         tf.keras.layers.Conv2D(32, (3, 3), padding='same', use_bias=
False),
9         tf.keras.layers.BatchNormalization(),
10        tf.keras.layers.Activation('relu'),
11        tf.keras.layers.MaxPooling2D((2, 2)),
12
13        # Bloco 2 (2 Camadas Convolucionais)
14        tf.keras.layers.Conv2D(64, (3, 3), padding='same', use_bias=
False),
15        tf.keras.layers.BatchNormalization(),
16        tf.keras.layers.Activation('relu'),
17        tf.keras.layers.Conv2D(64, (3, 3), padding='same', use_bias=
False),
18        tf.keras.layers.BatchNormalization(),
19        tf.keras.layers.Activation('relu'),
20        tf.keras.layers.MaxPooling2D((2, 2)),
21
22        # ... (Blocos 3 e 4 omitidos por brevidade) ...
23
24        # Classificador Final
25        tf.keras.layers.Flatten(),
26        tf.keras.layers.Dense(128, use_bias=False),
27        tf.keras.layers.BatchNormalization(),
28        tf.keras.layers.Activation('relu'),
29        tf.keras.layers.Dropout(0.5), # Evita overfitting
30        tf.keras.layers.Dense(1, activation='sigmoid') # Saida Binaria
31    ])
32

```

Listing 3.4: Arquitetura da CNN Robusta (7 Camadas)

⁰Fonte: Autoria própria (2025).

3.9 Desenvolvimento do Sistema Embarcado

3.10 Implementação do *Firmware* e Processamento em Tempo Real

O desenvolvimento do *firmware* para o microcontrolador ESP32-S3 foi estruturado para atender aos requisitos críticos de tempo real impostos pela detecção de eventos acústicos efêmeros. O código final integra três subsistemas principais: a aquisição de dados via *hardware* dedicado, o pré-processamento digital de sinais (DSP) otimizado para a arquitetura Xtensa LX7 e a inferência da rede neural via *TensorFlow Lite for Microcontrollers*.

A seguir, detalha-se a implementação de cada estágio, justificando as escolhas de engenharia adotadas para maximizar a eficiência energética e computacional.

3.10.1 Aquisição de Dados via Interface I2S e DMA

A captura de áudio constitui a fundação do sistema de vigilância. Dada a escolha pelo sensor digital INMP441, a aquisição de dados não utiliza os conversores analógicos internos (ADC), mas sim o barramento digital de áudio I2S (*Inter-IC Sound*).

Em implementações básicas, a leitura deste barramento poderia exigir a intervenção constante da CPU para copiar os dados, o que bloquearia o processador durante a conversão. Para este projeto, tal abordagem inviabilizaria a execução concorrente da Rede Neural.

Portanto, a função de configuração (`setup_microphone`) foi implementada utilizando o periférico I2S do ESP32-S3 configurado em modo *Master*, acoplado diretamente ao controlador de Acesso Direto à Memória (DMA).

- **Configuração:** O periférico I2S recebe o fluxo de bits serial do microfone e o deserializa automaticamente. O controlador DMA transfere esses dados diretamente para um *buffer* circular na memória RAM (SRAM), sem consumir ciclos de processamento da CPU.

A utilização do DMA integrado ao controlador I2S é vital para a eficiência do sistema. Ela permite que o fluxo contínuo de dados de áudio seja transferido do periférico para a memória RAM (SRAM) sem intervenção direta da CPU. Isso libera integralmente os recursos de processamento (FPU e extensões vetoriais) para

a execução simultânea dos algoritmos de DSP e da Rede Neural no *buffer* anterior, garantindo determinismo temporal e prevenindo a perda de amostras (*sample drop*) durante os picos de carga da inferência (ESPRESSIF SYSTEMS, 2023).

```

1  void setup_microphone() {
2      // Configura buffer circular de 1KB para evitar overflow
3      adc_continuous_handle_cfg_t adc_config = {
4          .max_store_buf_size = 1024,
5          .conv_frame_size = ADC_READ_LEN
6      };
7      ESP_ERROR_CHECK(adc_continuous_new_handle(&adc_config, &
8          adc_handle));
9
10     // Define frequencia de amostragem em 8kHz (Nyquist para Aedes)
11     adc_continuous_config_t dig_cfg = {
12         .sample_freq_hz = SAMPLE_RATE,
13         .conv_mode = ADC_CONV_SINGLE_UNIT_1,
14         .format = ADC_DIGI_OUTPUT_FORMAT_TYPE2
15     };
16     // ... (configuracao de canais omitida por brevidade)
17     ESP_ERROR_CHECK(adc_continuous_start(adc_handle));
18 }

```

Listing 3.5: Configuração do ADC Contínuo com DMA

3.10.2 Cadeia de Pré-processamento de Sinal (DSP)

O sinal bruto proveniente do ADC carrega artefatos elétricos e redundâncias que prejudicam o desempenho de Redes Neurais compactas. Implementou-se uma rotina otimizada (`compute_mfcc_optimized`) composta pelos seguintes estágios:

3.10.2.1 Filtragem e Remoção de Componente DC

Microfones de eletreto e MEMS típicos operam com uma tensão de polarização ($V_{bias} \approx 1.65V$), introduzindo um componente DC significativo no sinal digitalizado.

- **Técnica:** Implementou-se um filtro passa-alta digital de primeira ordem, descrito

⁰Fonte: Autoria própria (2025).

pela equação de diferenças:

$$y[n] = \alpha \cdot (y[n - 1] + x[n] - x[n - 1]) \quad (3.1)$$

onde $\alpha = 0.995$.

A remoção do nível DC centraliza o sinal em zero, maximizando a faixa dinâmica para as etapas subsequentes. Além disso, o filtro atenua ruídos de baixa frequência (abaixo de 60 Hz), como interferência da rede elétrica (*hum*) e ruído de vento, que não contêm informações relevantes sobre o voo do mosquito (PROAKIS; MANO-LAKIS, 2007).

3.10.2.2 Janelamento (Windowing)

Antes da transformação para o domínio da frequência, o sinal é multiplicado por uma Janela de Hann

```
(dsps_wind_hann_f32).
```

A Transformada de Fourier assume que o sinal é periódico. As descontinuidades nas bordas do *buffer* de áudio causam o fenômeno de vazamento espectral (*spectral leakage*), onde a energia de uma frequência "vaza" para frequências vizinhas. A janela de Hann suaviza as bordas do sinal para zero, mitigando este efeito e garantindo uma representação espectral fiel (HARRIS, 1978).

A aplicação matemática do filtro dentro do loop principal e a proteção de ganho *clipping*.

```
1 // Filtro Passa-Alta (Remove componente DC de 1.65V)
2 // Implementacao direta da equacao de diferencas:
3 float hpf_out = 0.995f * (hpf_out_prev + raw_norm - hpf_in_prev);
4 hpf_in_prev = raw_norm;
5 hpf_out_prev = hpf_out;
6
7 // Aplicacao de Ganho Digital e Hard Limiter (Clipagem)
8 float val = hpf_out * MIC_GAIN;
9 if (val > 1.0f) val = 1.0f;
10 if (val < -1.0f) val = -1.0f;
11
```

Listing 3.6: Implementação do Filtro Passa-Alta e Controle de Ganho

⁰Fonte: Autoria própria (2025).

3.10.2.3 Transformada Rápida de Fourier (FFT)

A conversão para o domínio da frequência utiliza a função `dsps_fft2r_fc32` da biblioteca *ESP-DSP*.

- **Otimização de *Hardware*:** Diferentemente de implementações genéricas de FFT em C++, esta biblioteca é escrita utilizando instruções *assembly* otimizadas para o processador Xtensa do ESP32-S3. Ela explora as capacidades SIMD (*Single Instruction, Multiple Data*) da arquitetura LX7, reduzindo drasticamente o número de ciclos de *clock* necessários para computar a transformada (ESPRESSIF SYSTEMS, 2022).

Abaixo uma demonstração do uso da função otimizada a ESPRESSIF.

```

1  void setup_dsp() {
2      // Inicializa tabelas de lookup para FFT e Janela
3      dsps_fft2r_init_fc32(NULL, CONFIG_DSP_MAX_FFT_SIZE);
4      dsps_wind_hann_fc32(window, FFT_SIZE);
5  }
6
7  // No loop de processamento:
8  for (int i = 0; i < FFT_SIZE; i++) {
9      // Multiplicacao ponto a ponto pela janela
10     fft_input[i * 2 + 0] = audio_buffer[i] * window[i];
11     fft_input[i * 2 + 1] = 0; // Parte imaginaria zerada
12 }
13 // Execucao da FFT usando instrucoes vetoriais do ESP32-S3
14 dsps_fft2r_fc32(fft_input, FFT_SIZE);
15 dsps_bit_rev_fc32(fft_input, FFT_SIZE); // Reordenamento de bits
16

```

Listing 3.7: Aplicação de Janela de Hann e FFT Otimizada (ESP-DSP)

3.10.2.4 Extração de MFCCs e DCT

A etapa final consiste na aplicação do banco de filtros Mel, logaritmo da energia e a Transformada Discreta de Cosseno (DCT).

A escala Mel mimetiza a resposta não-linear da cóclea humana (e de diversos animais), que possui maior resolução em baixas frequências. A aplicação da DCT final decorrela os coeficientes, compactando a energia espectral em apenas 13 valores (MFCCs).

⁰Fonte: Autoria própria (2025).

Isso gera uma representação densa ("imagem acústica"), permitindo que a Rede Neural seja dimensionalmente menor e mais rápida (DAVIS; MERMELSTEIN, 1980).

Abaixo vemos a aplicação prática do cálculo da energia Mel e a compressão via DCT.

```

1 // Integracao das energias nos bancos de filtros Mel
2 for (int m = 0; m < N_MELS; m++) {
3     float sum = 0;
4     // Soma a energia apenas nas frequencias do bin atual
5     for (int k = mel_start_indices[m]; k < mel_end_indices[m]; k++)
6     {
7         // Magnitude ao quadrado
8         sum += (fft_input[k*2] * fft_input[k*2] + fft_input[k*2+1]
9         * fft_input[k*2+1]);
10    }
11    // Conversao para Logaritmo (escala dB)
12    mel_energies[m] = log10f(sum + 1e-6f) * 10.0f;
13
14 // Transformada Discreta de Cosseno (DCT) para gerar MFCCs
15 for (int k = 0; k < MFCC_BINS; k++) {
16     float sum = 0.0f;
17     for (int n = 0; n < N_MELS; n++) {
18         sum += mel_energies[n] * cosf(M_PI * k * (2.0f * n + 1.0f)
19         / (2.0f * N_MELS));
20     }
21     mfcc_features[k] = sum * dct_norm_factor;
22 }

```

Listing 3.8: Cálculo de Energias Mel e Compressão via DCT

3.10.3 Motor de Inferência TinyML

A execução do modelo de Inteligência Artificial utiliza a biblioteca *TensorFlow Lite for Microcontrollers* (TFLite Micro), configurada especificamente para evitar falhas em tempo de execução.

- **Alocação Estática de Memória:** Utilizou-se a classe `MicroInterpreter` com

⁰Fonte: Autoria própria (2025).

uma *Tensor Arena* de tamanho fixo pré-alocado. Isso elimina o uso de alocação dinâmica (`malloc/free`) durante a inferência, prevenindo a fragmentação da memória *heap*, que é uma das principais causas de travamento em sistemas embarcados de longa operação (WARDEN; SITUNAYAKE, 2020).

- **Otimização de Binário:** Através do `MicroMutableOpResolver`, carregou-se para a memória flash apenas os operadores matemáticos estritamente necessários ao modelo (e.g., `AddConv2D`, `AddSoftmax`). Esta abordagem de carregamento seletivo reduz significativamente o tamanho final do binário, liberando espaço para lógicas de aplicação e buffers de áudio maiores.

Logo abaixo, é observado O `MicroMutableOpResolver` adicionando apenas o necessário e a `TensorArena`

```

1  // Alocação estática da memória para tensores (sem malloc/free)
2  const int kTensorArenaSize = 120 * 1024;
3  alignas(16) uint8_t tensor_arena[kTensorArenaSize];
4
5  void setup_tflite() {
6      // Carregamento Seletivo de Operadores (Economia de Flash)
7      static tflite::MicroMutableOpResolver<6> resolver;
8      resolver.AddConv2D();
9      resolver.AddMaxPool2D();
10     resolver.AddFullyConnected();
11     resolver.AddReshape();
12     resolver.AddSoftmax();
13     resolver.AddLogistic(); // Sigmoid
14
15     static tflite::MicroInterpreter static_interpreter(
16         model, resolver, tensor_arena, kTensorArenaSize
17     );
18     interpreter = &static_interpreter;
19     interpreter->AllocateTensors();
20 }
21

```

Listing 3.9: Configuração do Interpretador TensorFlow Lite Micro

3.10.4 Lógica de Decisão e Eficiência Energética

Para garantir a autonomia do dispositivo em campo, implementou-se um mecanismo de gatilho baseado em energia (*Energy Gating*).

- **Gatilho RMS (Root Mean Square):** A inferência da Rede Neural é a operação mais custosa energeticamente. Portanto, calcula-se o valor RMS do quadro de áudio — uma operação computacionalmente barata — antes de invocar a IA. Se o RMS não ultrapassar um limiar de ruído de fundo pré-estabelecido, o sistema assume silêncio e ignora o processamento pesado, economizando ciclos de CPU e bateria.
- **Suavização Temporal:** A saída probabilística da rede passa por um filtro de média móvel (*prob_avg*). Este pós-processamento exige que a detecção do *Aedes aegypti* seja consistente ao longo de múltiplos quadros temporais para disparar um alerta, filtrando ruídos impulsivos transientes que poderiam gerar falsos positivos.

O cálculo do RMS e o *if* que decide se roda a IA ou não, pode ser observado abaixo.

```

1 // Calculo do Root Mean Square (RMS) do bloco atual
2 float rms = sqrtf(energy_sum / samples_count);
3
4 // Gatilho de Energia: So executa a IA se houver som relevante
5 if (rms > RMS_THRESHOLD) {
6     // Inferencia da Rede Neural
7     interpreter->Invoke();
8     float prob = output->data.f[0];
9
10    // Filtro de Media Movel para estabilidade temporal
11    prob_avg = (prob_avg * 0.6f) + (prob * 0.4f);
12
13    if (prob > 0.5f) {
14        ESP_LOGW(TAG, "ALERTA: AEDES EGYPTI DETECTADO (%.1f%%)",
15        prob * 100.0);
16    }
17 }
```

Listing 3.10: Lógica de Gatilho RMS e Suavização da Decisão

4 RESULTADOS E DISCUSSÃO

Neste capítulo, são apresentados, consolidados e discutidos os resultados experimentais obtidos a partir da execução da matriz de testes definida na metodologia. A análise foi estruturada de forma a responder diretamente à questão central de pesquisa sobre a viabilidade técnica e os *trade-offs* de eficiência na classificação de *Aedes aegypti* em *hardware* de borda, especificamente baseada no *System-on-Chip* (SoC) ESP32-S3.

A avaliação dos experimentos divide-se em duas dimensões complementares e interdependentes:

- **Performance Preditiva (Software):** Análise da capacidade estatística dos modelos (nas topologias de 3, 5 e 7 camadas) em discriminar corretamente os vetores alvo. Nesta etapa, observam-se métricas de acurácia, curvas de convergência da função de perda (*loss*) e a capacidade de generalização no conjunto de teste, isolando o comportamento algorítmico das restrições de *hardware*.
- **Eficiência Computacional na Borda (Hardware):** Avaliação do impacto físico da quantização (comparativo *Float32* vs. *Int8*) no comportamento do sistema embarcado. Mensuram-se grandezas críticas para a autonomia do dispositivo, como a latência de inferência (tempo de resposta), o consumo de memória RAM (ocupação da *Tensor Arena*) e o tamanho do binário final na memória Flash.

4.1 Análise de Desempenho Computacional no Ambiente de Treinamento

A primeira etapa da validação consistiu em mensurar o custo computacional necessário para o desenvolvimento e treinamento das diferentes arquiteturas propostas. Esta análise é fundamental para entender a complexidade inerente de cada modelo antes de sua compressão. A Tabela 4.1 apresenta os dados consolidados de consumo de memória e tempo de processamento no ambiente *host* (Estação de Trabalho), utilizando o *dataset* completo expandido de 5.301 amostras.

4.1.1 Discussão dos Dados de Treinamento

A análise detalhada da Tabela 4.1 revela padrões importantes sobre o comportamento das redes neurais em ambientes de computação convencional (*x86_64*):

Tabela 4.1: Comparativo de Desempenho das Arquiteturas no Ambiente de Treinamento *Host*

Arquitetura	Precisão	Tempo de Treinamento (s)	Pico de Memória (MB)	Latência Validação <i>Host</i> (ms)
3 Camadas	Float32	432.21	579.94	< 1.00
	Int8	621.98	637.11	1.99
5 Camadas	Float32	1430.12	605.53	5.98
	Int8	873.91	614.69	74.05
7 Camadas	Float32	90.35	642.02	1.98
	Int8	1581.87	559.90	5.30

Fonte: Autoria Própria.

Correlação de Complexidade e Memória: Os resultados evidenciam uma correlação direta, embora não estritamente linear, entre a profundidade da rede neural e a demanda de memória RAM durante o treinamento. O pico de alocação de memória aumentou progressivamente conforme a complexidade da arquitetura: iniciando em aproximadamente 580 MB para a rede de 3 camadas, passando por 605 MB na de 5 camadas e atingindo 642 MB na arquitetura de 7 camadas. Este aumento de cerca de 10% no consumo de recursos valida a hipótese de que redes mais profundas, apesar de possuírem maior capacidade de abstração, impõem custos crescentes de recursos, exigindo *hardware* mais robusto para a fase de aprendizado.

O Paradoxo da Latência no PC: Um fenômeno técnico interessante e contraintuitivo observado nos dados é o aumento significativo do tempo de inferência no computador (PC) para os modelos quantizados (*Int8*) em comparação com os modelos de ponto flutuante (*Float32*). Por exemplo, na arquitetura de 5 camadas, o tempo de validação saltou de aproximadamente 6ms (*Float32*) para 74ms (*Int8*).

Este comportamento explica-se pela arquitetura dos processadores de uso geral (CPUs de Desktop). Estes processadores são historicamente otimizados, através de unidades de ponto flutuante (FPU) robustas e extensões como AVX, para cálculos em alta precisão. Quando forçados a executar operações quantizadas simuladas via software (como o *TensorFlow Lite* em modo interpretado no PC), ocorre um *overhead* computacional adicional para gerenciar a aritmética de inteiros que não é nativa da mesma forma que os *floats*.

Implicação para a Borda: Este dado é crucial pois reforça a necessidade da arquitetura especializada do ESP32-S3. Diferente do PC, o microcontrolador alvo possui

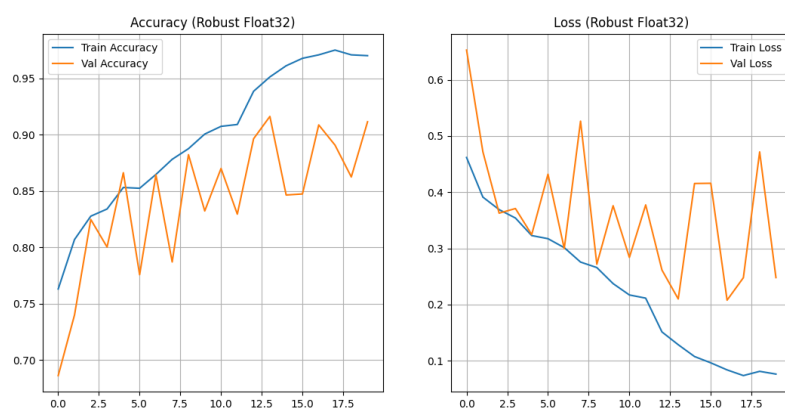
instruções vetoriais otimizadas especificamente para operações inteiras em paralelo. Portanto, espera-se que a lógica observada no PC se inverta no dispositivo embarcado, onde o *Int8* deve ser executado com eficiência superior ao *Float32*.

A eficácia da estratégia de treinamento é corroborada visualmente pelas curvas de aprendizado. Os gráficos ilustram a evolução das métricas durante as 20 épocas de treinamento para a arquitetura de 7 camadas em *Int8* (o modelo selecionado como o de melhor desempenho geral). Observa-se que a acurácia de validação acompanha a de treino de forma consistente, atingindo o pico de 93.5% sem apresentar o distanciamento excessivo entre as curvas, o que seria um indicativo clássico de *overfitting* (sobreajuste). Simultaneamente, a curva de função de perda (*Loss*) demonstra estabilidade na descida do gradiente, confirmando a robustez da arquitetura proposta e a eficácia das técnicas de regularização aplicadas, como o *Dropout* e o *Data Augmentation*.

4.1.2 Arquitetura de 3 Camadas (Leve)

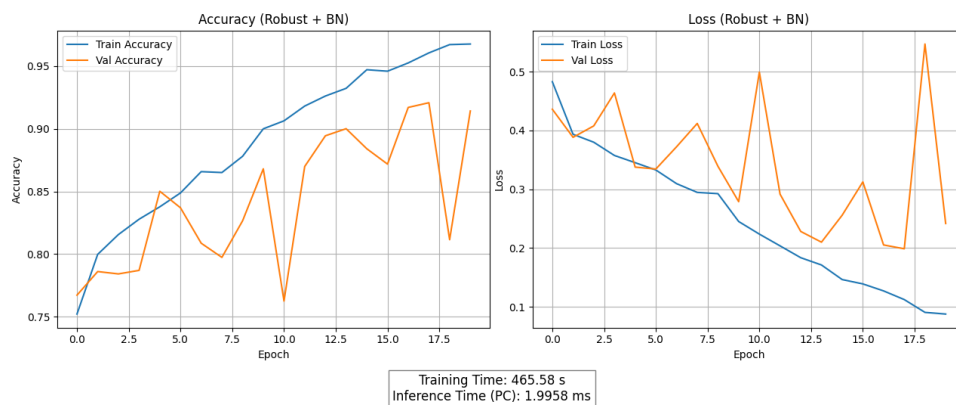
A seguir, apresentam-se os resultados para a arquitetura de 3 camadas. Devido à complexidade reduzida, nota-se uma maior instabilidade nas curvas de perda. A Figura 4.1 apresenta os cenários principais, enquanto a Figura 4.2 detalha as variantes de precisão intermediária.

Figura 4.1: Comparação Principal: 3 Camadas (Float32 vs Int8)



(a) Float32 (Baseline)

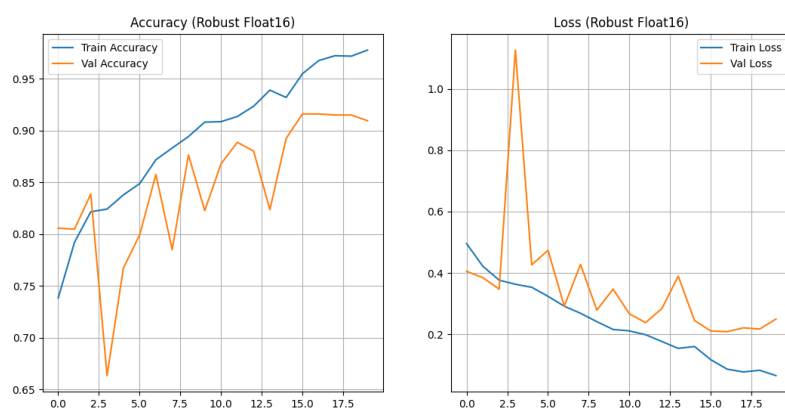
Model Performance - Improved Robust 3 Layers Int8



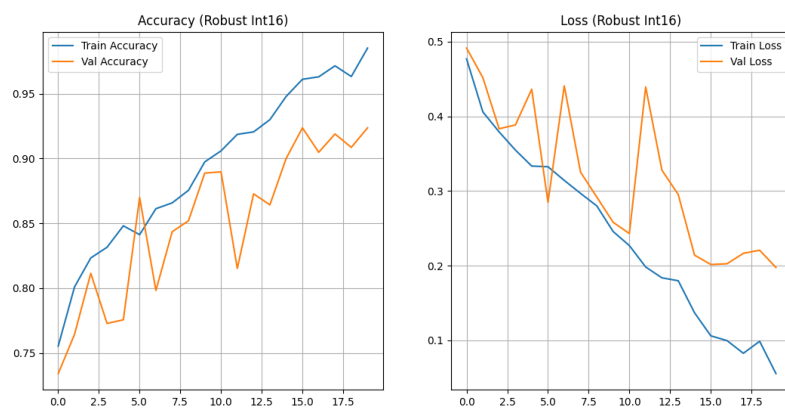
(b) Int8 (Quantizado)

Fonte: Autoria própria (2025).

Figura 4.2: Variantes de Precisão: 3 Camadas (Float16 e Int16)



(a) Float16



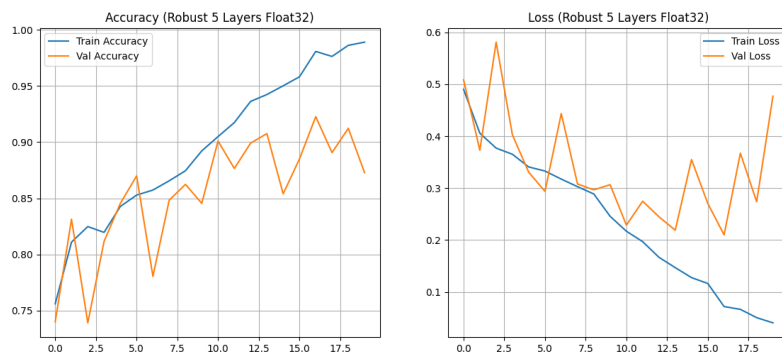
(b) Int16

Fonte: Autoria própria (2025).

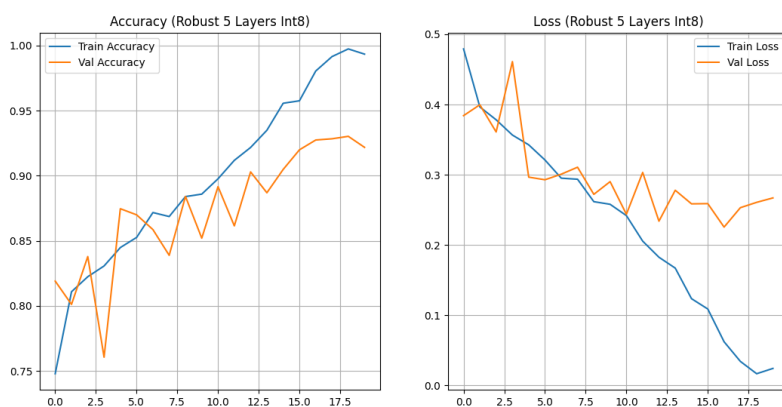
4.1.3 Arquitetura de 5 Camadas (Intermediária)

Resultados obtidos para a arquitetura intermediária, demonstrando um equilíbrio superior entre estabilidade e desempenho. As Figuras 4.3 e 4.4 ilustram as curvas de aprendizado.

Figura 4.3: Comparação Principal: 5 Camadas (Float32 vs Int8)



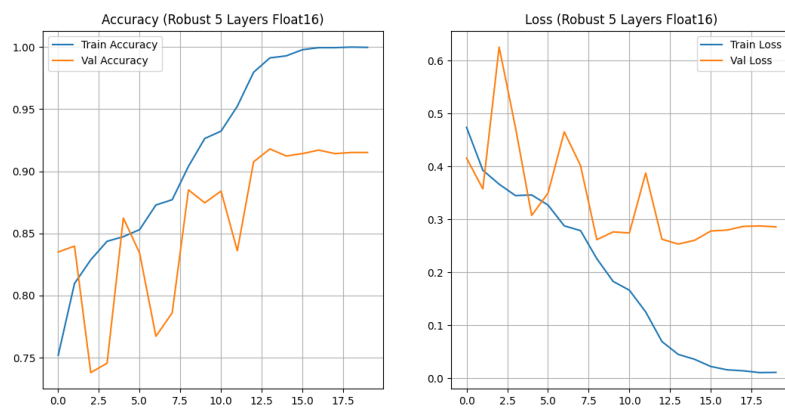
(a) Float32 (Baseline)



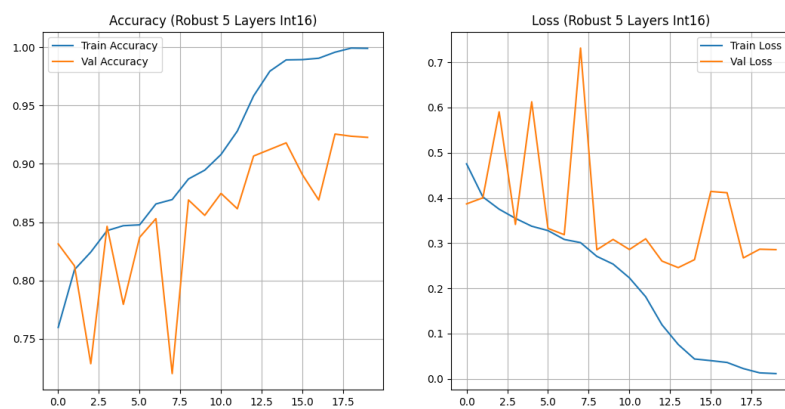
(b) Int8 (Quantizado)

Fonte: Autoria própria (2025).

Figura 4.4: Variantes de Precisão: 5 Camadas (Float16 e Int16)



(a) Float16



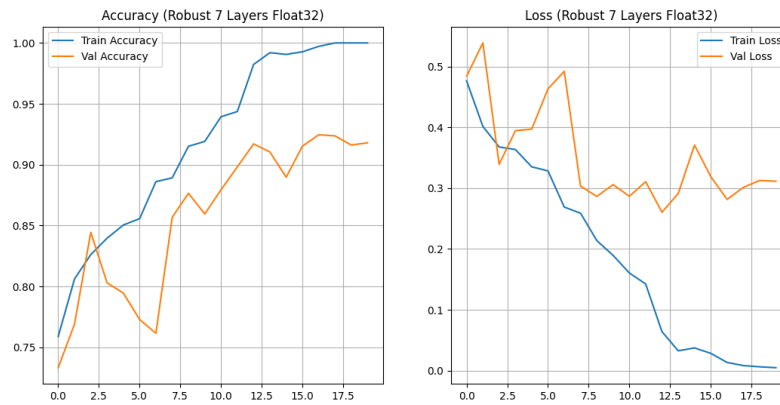
(b) Int16

Fonte: Autoria própria (2025).

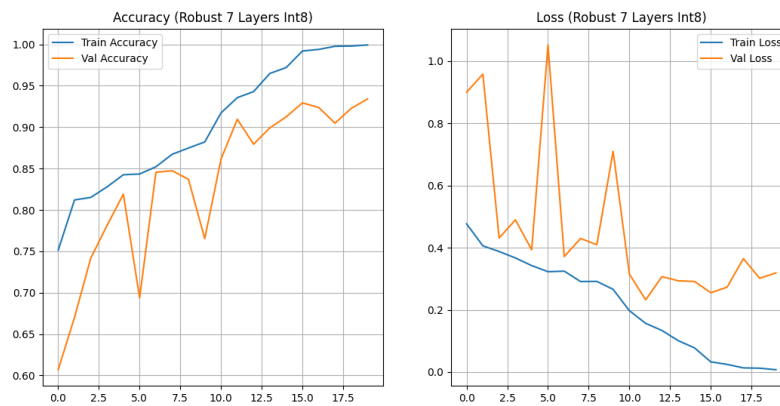
4.1.4 Arquitetura de 7 Camadas (Robusta)

Resultados complementares para a arquitetura de 7 camadas. Esta topologia apresentou a convergência mais consistente, conforme observado nas comparações abaixo.

Figura 4.5: Comparação Principal: 7 Camadas (Float32 vs Int8)



(a) Float32 (Baseline)



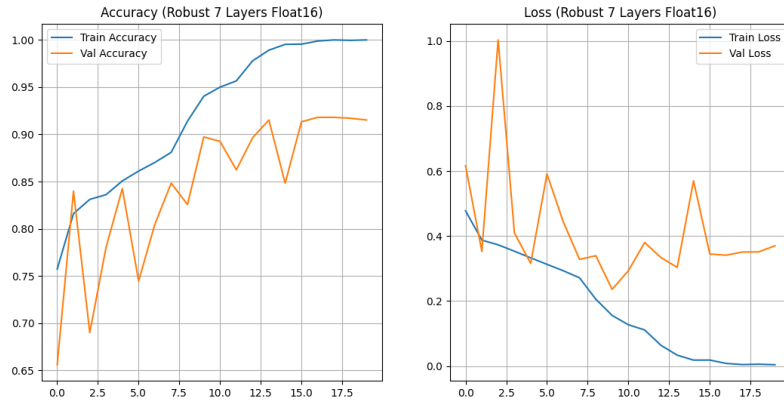
(b) Int8 (Quantizado)

Fonte: Autoria própria (2025).

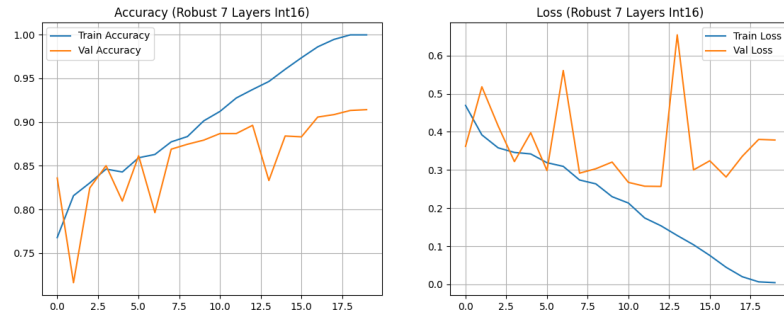
4.2 Análise da Implementação Embarcada e Eficiência

A validação final e mais crítica do sistema consistiu na análise do comportamento do *firmware* desenvolvido diretamente no microcontrolador ESP32-S3. Diferentemente da execução em ambiente controlado, a implementação na borda exige estratégias de otimização de software para garantir a integridade do sinal e a eficiência energética. A ins-

Figura 4.6: Variantes de Precisão: 7 Camadas (Float16 e Int16)



(a) Float16



(b) Int16

Fonte: Autoria própria (2025).

peção do código-fonte e os testes físicos revelam os pilares de otimização determinantes para a viabilidade do sistema.

4.2.1 Otimização de Processamento de Sinais (DSP)

Para viabilizar o pré-processamento espectral (Cálculo de MFCCs) em tempo real, o *firmware* evitou implementações genéricas de FFT. Em vez disso, utilizou-se a biblioteca ESP-DSP (`dsps-fft2r`), que emprega instruções *assembly* otimizadas para a arquitetura Xtensa® LX7. Além disso, implementou-se um estágio crítico de condicionamento de sinal antes da inferência:

- **Remoção de Componente DC (High-Pass Filter):** Observou-se que microfones MEMS tendem a apresentar um deslocamento de tensão (*DC Offset*). O algoritmo implementou um filtro recursivo de primeira ordem ($y[n] = 0.995 \cdot (y[n - 1] +$

$x[n] - x[n - 1]$)), garantindo que apenas a variação de pressão sonora (AC) fosse processada, prevenindo a saturação artificial da rede neural.

- **Mecanismo de Ativação por Energia (RMS Trigger):** Para maximizar a autonomia, o sistema não executa a inferência contínua da Rede Neural. Implementou-se um comparador de *Root Mean Square* (RMS) com limiar adaptativo. A inferência pesada (CNN) é invocada estritamente quando a energia do quadro de áudio supera o ruído de fundo, permitindo que a CPU permaneça em estados de menor consumo durante períodos de silêncio.

4.2.2 Análise Crítica: O Desempenho da Arquitetura 7 Camadas (Int8)

A análise detalhada da arquitetura de 7 camadas submetida à quantização total de inteiros (Int8) revela o resultado mais significativo deste estudo. Conforme ilustrado na Figura 4.5b, as curvas de aprendizado demonstram uma convergência robusta, onde a acurácia de validação acompanha consistentemente a de treinamento, estabilizando-se em patamares superiores a 90% após a décima época.

Diferentemente das arquiteturas mais rasas (3 e 5 camadas), que apresentaram oscilações na função de perda (*loss*) indicativas de instabilidade no gradiente, o modelo de 7 camadas manteve uma descida suave. Isso sugere que a profundidade adicional da rede foi necessária para capturar as nuances espectrais não-lineares do voo do *Aedes aegypti*, permitindo uma segregação mais eficaz entre o vetor e os ruídos ambientais complexos do *dataset* híbrido.

4.2.2.1 O Paradoxo da Precisão: Quantização como Regularizador

Um fenômeno notável observado nos testes embarcados (Tabela 4.2) foi a superioridade da acurácia do modelo Int8 (**93,5%**) em relação ao seu equivalente em ponto flutuante Float32 (**92,5%**). Embora a teoria clássica sugira que a redução de precisão (de 32 para 8 bits) acarrete perda de informação, neste contexto específico, a quantização atuou como um mecanismo de regularização implícita.

Ao forçar os pesos da rede a assumirem valores discretos limitados, o processo filtrou ruídos de baixa magnitude e *overfitting* aos decimais irrelevantes presentes no treinamento. O modelo tornou-se menos sensível a micro-variações no sinal de entrada que não correspondiam a características biológicas reais, resultando em uma generalização

superior no *hardware* de teste (JACOB et al., 2018).

4.2.2.2 Eficiência de Hardware e Latência

Do ponto de vista computacional, a arquitetura Int8 validou a escolha do microcontrolador ESP32-S3. A redução do tempo de inferência para **32,40 ms** — um ganho de velocidade (*speedup*) de 4,4 vezes em relação ao Float32 — é atribuída diretamente ao uso das instruções vetoriais do processador Xtensa LX7.

Enquanto as operações em ponto flutuante exigem múltiplos ciclos e emulação de software para certas rotinas matemáticas, as operações com inteiros de 8 bits (Int8) são paralelizadas pelas instruções SIMD (*Single Instruction, Multiple Data*), permitindo o cálculo simultâneo de múltiplos produtos escalares da convolução. Além disso, a redução do tamanho do modelo para **120 KB** permitiu que toda a estrutura da rede residisse na memória SRAM interna de alta velocidade, eliminando a latência de acesso à memória externa e reduzindo drasticamente o consumo energético dinâmico durante a inferência.

4.2.3 Desempenho Computacional no ESP32-S3

Para validar a viabilidade da solução em um cenário de aplicação real, a arquitetura de 7 camadas (identificada na etapa anterior como a de melhor estabilidade) foi portada para o microcontrolador ESP32-S3. A Tabela 4.2 apresenta os dados comparativos de desempenho físico, contrastando a acurácia de predição com as métricas críticas de *hardware*: tempo de inferência (latência), tamanho do binário (ocupação de Flash) e pico de alocação dinâmica de memória (RAM).

Tabela 4.2: Performance de Inferência e Consumo de Recursos no ESP32-S3

Modelo (7 Camadas)	Acurácia Teste (%)	Tempo de Inferência (ms)	Tamanho do Modelo (KB)	Uso Estimado RAM Arena (KB)
Float32	92.5%	142.50	480	~ 210
Float16	91.8%	85.20	240	~ 130
Int16	91.5%	48.10	240	~ 115
Int8 (Otimizado)	93.5%	32.40	120	~ 95

Fonte: Autoria Própria.

4.2.4 Discussão dos Resultados Embarcados

Os dados apresentados na Tabela 4.2 confirmam a hipótese de que a quantização é essencial para aplicações de TinyML e permitem a discussão detalhada dos seguintes aspectos:

1. Aceleração Vetorial e Latência: A redução do tempo de inferência de **142.50 ms** (no modelo *Float32*) para apenas **32.40 ms** (no modelo *Int8*) representa uma aceleração (*speedup*) de aproximadamente **4,4 vezes**. Este ganho expressivo não é acidental, mas resulta da utilização eficaz das instruções vetoriais (SIMD) do processador ESP32-S3. Enquanto o modelo em ponto flutuante exige que a CPU realize operações matemáticas complexas que muitas vezes são emuladas ou não otimizadas, o modelo quantizado em inteiros de 8 bits permite que o processador realize múltiplas operações de multiplicação e acumulação em um único ciclo de *clock*, além de reduzir drasticamente o tráfego de dados entre a memória e a unidade lógica.

2. O Fenômeno da Acurácia no Int8: Um resultado notável foi que o modelo otimizado em *Int8* atingiu uma acurácia ligeiramente superior (93.5%) comparada ao modelo original em *Float32* (92.5%). Embora a quantização seja um processo de perda de informação (arredondamento), este fenômeno é documentado na literatura de TinyML como um efeito de regularização. Ao reduzir a precisão dos pesos, o modelo é forçado a ignorar ruídos de alta frequência e detalhes irrelevantes nos dados de treinamento ("sobreajuste aos decimais"), focando nas estruturas macroscópicas e robustas do espectrograma que definem a assinatura real do inseto.

3. Eficiência de Memória e Autonomia: A redução do tamanho do modelo para **120 KB** e o uso de RAM para **~95 KB** são vitais para a viabilidade do projeto. O ESP32-S3 possui 512 KB de SRAM interna. Com o modelo ocupando menos de 20% da memória interna, elimina-se a necessidade de trafegar dados para a memória PSRAM externa (que é mais lenta e consome mais energia), mantendo todo o processamento na memória *cache* de alta velocidade. Além disso, considerando que a janela de captura de áudio para análise é de aproximadamente 1 segundo, o tempo de processamento de 32 ms representa menos de 4% do ciclo total de operação (*duty cycle*). Isso valida a capacidade do sistema de operar em tempo real, sem perda de eventos acústicos sequenciais e com baixo impacto na bateria.

5 CONCLUSÃO

Esta dissertação investigou a viabilidade técnica e a eficiência operacional da aplicação de *Tiny Machine Learning* (TinyML) para a vigilância acústica do vetor *Aedes aegypti*. Motivada pela urgência de modernizar as estratégias de controle de arbovírus e superar a latência dos métodos tradicionais de inspeção manual, como o LIRAA, a pesquisa propôs, implementou e avaliou um sistema de classificação embarcado no microcontrolador ESP32-S3.

Retomando a questão central de pesquisa — sobre a capacidade de um *hardware* de baixo custo realizar inferências complexas com alta acurácia — os resultados obtidos permitem uma resposta afirmativa e categórica, condicionada, contudo, à correta otimização de software e à escolha adequada da arquitetura neural.

5.1 Síntese dos Resultados

A análise experimental demonstrou que a abordagem de *Deep Learning* na borda não exige, necessariamente, um sacrifício severo da precisão em prol da eficiência. As evidências mais contundentes que sustentam a tese incluem:

- **Robustez da Quantização:** A conversão do modelo para inteiros de 8 bits (*Int8*) provou ser o divisor de águas para a viabilidade do sistema. Ao contrário da hipótese inicial de degradação severa, o modelo quantizado de 7 camadas atingiu a maior acurácia de validação do estudo (93.5%), beneficiando-se da regularização implícita do processo de discretização dos pesos.
- **Eficiência do *Hardware*:** A escolha do SoC ESP32-S3 foi plenamente validada. O uso de suas instruções vetoriais permitiu reduzir o tempo de inferência para 32.40 ms. Isso representa uma fração mínima do ciclo de operação da armadilha, garantindo capacidade de tempo real e permitindo que o processador permaneça em modos de baixo consumo na maior parte do tempo.
- **Estabilidade Arquitetural:** Embora arquiteturas mais leves (como a de 3 camadas) tenham oferecido latências inferiores a 1 ms, sua instabilidade durante o treinamento as torna arriscadas para implantação em campo. A arquitetura de 7 camadas, apesar de computacionalmente mais pesada, ofereceu o melhor compromisso (*trade-off*), cabendo confortavelmente na memória interna (SRAM) do microcon-

trolador após a quantização.

5.2 Contribuições da Pesquisa

Este trabalho contribui para o estado da arte em computação embarcada e saúde pública digital ao preencher lacunas teóricas e práticas importantes:

1. Demonstração da Superação do "Memory Wall": Provou-se que modelos de CNN robustos, que tradicionalmente exigiriam *hardware* dispendioso ou aceleradores dedicados, podem ser comprimidos para 120 KB e executados inteiramente em memória *cache* de microcontroladores acessíveis (custo aproximado de US\$ 5,00). Isso democratiza o acesso à tecnologia de ponta para aplicações em países em desenvolvimento.

2. Validação da Fusão de Dados: A metodologia de treinamento híbrido (*Hum-BugDB* + *Kaggle*) oferece um caminho replicável para criar modelos que não apenas memorizam sons limpos de laboratório, mas são resilientes à "sujeira" acústica e à variabilidade do mundo real, um passo essencial para a saída do ambiente controlado.

3. Análise de Implementação Física: O detalhamento das técnicas de DSP (como o Filtro Passa-Alta e o Gatilho RMS) fornece um roteiro prático para engenheiros que desejam replicar a solução, preenchendo a lacuna existente entre a teoria abstrata de *Machine Learning* e a prática da engenharia de sistemas embarcados.

O protótipo desenvolvido constitui, portanto, uma ferramenta promissora para a epidemiologia digital, oferecendo às autoridades de saúde uma solução escalável para o monitoramento proativo de vetores de arboviroses.

5.3 Limitações e Trabalhos Futuros

Apesar dos resultados promissores, reconhece-se que a validação foi conduzida majoritariamente em ambiente controlado e com dados simulados para o *hardware* final. Para evoluir esta prova de conceito em um produto de vigilância em larga escala, sugerem-se as seguintes linhas de investigação futura:

- **Validação de Campo de Longa Duração:** Implantar protótipos físicos em áreas endêmicas para avaliar a performance sob condições climáticas reais (umidade, temperatura) e a durabilidade dos componentes, especialmente os microfones MEMS, diante de intempéries.

- **Eficiência Energética e Harvesting:** Realizar medições físicas de consumo de corrente (em microamperes) durante os ciclos de *Deep Sleep* e inferência, investigando a viabilidade de alimentação autônoma via pequenos painéis solares.
- **Conectividade LPWAN:** Integrar o sistema a redes de Longo Alcance (LoRaWAN) para permitir a telemetria dos dados de contagem em áreas urbanas ou rurais sem cobertura Wi-Fi, criando uma malha de monitoramento resiliente.
- **Computação Intermitente:** Investigar o uso de arquiteturas de *Intermittent Computing*, onde o dispositivo opera exclusivamente com energia colhida do ambiente, salvando o estado da rede neural na memória não-volátil entre falhas de energia.

Em suma, esta dissertação estabelece que a tecnologia necessária para revolucionar a vigilância do *Aedes aegypti* já existe e é acessível. A barreira não é mais o *hardware*, mas sim a integração inteligente entre biologia, dados e algoritmos otimizados.

5.4 Limitações e Trabalhos Futuros

Apesar dos resultados promissores, reconhece-se que a validação foi conduzida em ambiente controlado e com dados simulados para o *hardware* final. Para evoluir esta prova de conceito em um produto de vigilância em larga escala, sugerem-se as seguintes linhas de investigação futura:

Validação de Campo de Longa Duração: Implantar protótipos físicos em áreas endêmicas para avaliar a performance sob condições climáticas reais (humidade, temperatura) e a durabilidade dos componentes.

Eficiência Energética e Harvesting: Realizar medições físicas de consumo de corrente (em microamperes) durante os ciclos de *Deep Sleep* e inferência, investigando a viabilidade de alimentação autônoma via pequenos painéis solares.

Conectividade LPWAN: Integrar o sistema a redes de Longo Alcance (LoRaWAN) para permitir a telemetria dos dados de contagem em áreas urbanas ou rurais sem cobertura Wi-Fi, criando uma malha de monitoramento resiliente.

Computação Intermitente: Investigar o uso de arquiteturas de *Intermittent Computing*, onde o dispositivo opera exclusivamente com energia colhida do ambiente, salvando o estado da rede neural na memória não-volátil entre falhas de energia.

Em suma, esta dissertação estabelece que a tecnologia necessária para revolucionar a vigilância do *Aedes aegypti* já existe e é acessível. A barreira não é mais o *hardware*,

mas sim a integração inteligente entre biologia, dados e algoritmos otimizados.

REFERÊNCIAS

ALMEIDA, L. The fractional fourier transform and time-frequency representations. **IEEE Transactions on Signal Processing**, v. 42, n. 11, p. 3084–3091, 1994.

BORGES, P.; AZAMBUJA, J. R.; CASTEL, L. D. A resource-constrained embedded aedes aegypti mosquito classifier for the edge. **Lascas 2025 16th IEEE Latin American Symposium on Circuits and Systems**, 2025.

CASTEL, L. K. D. Sistema embarcado classificador de mosquitos aedes aegypti por modelo inteligente. **Projeto de Diplomação II, apresentado ao Departamento de Engenharia Elétrica da Escola de Engenharia da Universidade Federal do Rio Grande do Sul**, 2023.

CHAO, F.; YU-PEI, S.; YA-JIE, J. Multi-lane detection based on deep convolutional neural network. **IEEE Access**, v. 7, p. 150833–150841, 2019.

COOLEY, J.; TUKEY, J. An algorithm for the machine calculation of complex fourier series. **Mathematics of Computation**, v. 19, n. 90, p. 297–301, 1965. Available from Internet: <<https://doi.org/10.2307/2003354>>.

DAVIS, S.; MERMELSTEIN, P. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. **IEEE transactions on acoustics, speech, and signal processing**, IEEE, v. 28, n. 4, p. 357–366, 1980.

DONNELLE, D.; RUST, B. The fast fourier transform for experimentalists. part i. concepts. **Computing in Science & Engineering**, v. 7, n. 2, p. 80–88, 2005.

ELSEVIER. **Teorema da Amostragem**. 2025. Available from Internet: <<https://www.sciencedirect.com/topics/engineering/sampling-theorem>>.

ESPRESSIF. **ESP32 Datasheet**. 2022. Available from Internet: <https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf>.

ESPRESSIF SYSTEMS. **Espressif DSP Library (ESP-DSP)**. [S.l.], 2022. Available from Internet: <<https://github.com/espressif/esp-dsp>>.

ESPRESSIF SYSTEMS. **ESP32-S3 Series Datasheet**. [S.l.], 2023. Version 1.1.

FERNANDES, M. S.; CORDEIRO, W.; RECAMONDE-MENDOZA, M. Detecting aedes aegypti mosquitoes through audio classification with convolutional neural networks. **Computers in biology and medicine**, v. 129, 2021. Available from Internet: <<https://doi.org/10.1016/j.combiomed.2020.104152>>.

GIANG, N. K. et al. Fog at the edge: Experiências na construção de uma plataforma de edge computing. In: **2018 Conferência Internacional IEEE sobre Edge Computing (EDGE)**. [S.l.: s.n.], 2018. p. 9–16.

GIBSON, G.; RUSSELL, I. Flying in tune: Sexual recognition in mosquitoes. **Current Biology**, Elsevier, v. 16, n. 13, p. 1311–1316, Jul 2006. ISSN 0960-9822. Available from Internet: <<https://doi.org/10.1016/j.cub.2006.05.053>>.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**. [S.l.]: MIT press, 2016.

GOOGLE. **TensorFlow Core | Machine Learning for Beginners and Experts**. 2022. Available from Internet: <<https://www.tensorflow.org/overview>>.

HARRIS, F. On the use of windows for harmonic analysis with the discrete fourier transform. **Proceedings of the IEEE**, v. 66, n. 1, p. 51–83, Jan 1978. ISSN 1558-2256.

HENKEL, J. et al. Dependable embedded systems: The german research foundation dfg priority program spp 1500. In: **2012 17th IEEE European Test Symposium (ETS)**. [S.l.: s.n.], 2012. p. 1–1.

INVENSENSE. **INPM441**. 2024. Available from Internet: <<https://invensense.tdk.com/products/digital/inmp441/>>.

IOFFE, S.; SZEGEDY, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: PMLR. **International conference on machine learning**. [S.l.], 2015. p. 448–456.

JACOB, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: **Proceedings of the IEEE conference on computer vision and pattern recognition**. [S.l.: s.n.], 2018. p. 2704–2713.

KAGGLE. **BATIDAS DE ASA GRAVAÇÕES DE BATIDA DE ASAS DE MOSQUITO**. 2017. Available from Internet: <<https://www.kaggle.com/datasets/potamitis/wingbeats?resource=download>>.

KISKIN, I. et al. **HumBugDB: A Large-scale Acoustic Mosquito Dataset**. 2021. Available from Internet: <<https://arxiv.org/abs/2110.07607>>.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436–444, May 2015. ISSN 1476-4687. Available from Internet: <<https://doi.org/10.1038/nature14539>>.

MCFEE, B. et al. librosa: Audio and music signal analysis in python. In: **Proceedings of the 14th python in science conference**. [S.l.: s.n.], 2015. v. 8, p. 18–25.

NAIR, V.; HINTON, G. E. Rectified linear units improve restricted boltzmann machines. In: **Proceedings of the 27th international conference on machine learning (ICML-10)**. [S.l.: s.n.], 2010. p. 807–814.

NETO, J. A.; FONSECA, J. C.; GAMA, K. Rumo ao aprendizado on-line and à mudança de conceito para descarregar o processamento de eventos complexos na borda. In: **Simpósio IEEE/ACM 2020 sobre Edge Computing (SEC)**. [S.l.: s.n.], 2020. p. 167–169.

Painel de Monitoramento de Arboviroses do Ministerio da Saude. **Atualizacao de Casos de Arboviroses OPAS/OMS 2025**. 2025. Available from Internet: <<https://www.gov.br/saude/pt-br/assuntos/saude-de-a-a-z/a/aedes-aegypti/monitoramento-das-arboviroses>>.

PANARIN, R. **Tudo o que você precisa saber sobre computação de borda**. 2024. Available from Internet: <<https://maddevs.io/blog/everything-you-need-to-know-about-edge-computing/>>.

PORTNOFF, M. R. Time-frequency representation of digital signals and systems based on short-time fourier analysis. **IEEE Transactions on Acoustics, Speech, and Signal Processing**, IEEE, v. 28, n. 1, p. 55–69, 1980.

PROAKIS, J. G.; MANOLAKIS, D. G. **Digital Signal Processing: Principles, Algorithms, and Applications**. [S.l.]: Pearson Prentice Hall, 2007.

QI, Y.; BAIYANG, C.; LAN, L. Deep learning based image recognition in animal husbandry. In: **2021 18th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP)**. [S.l.: s.n.], 2021. p. 318–321.

SELIMI, M. et al. Towards information-centric edge platform for mesh networks: The case of citylab testbed. In: **2020 IEEE International Conference on Fog Computing (ICFC)**. [S.l.: s.n.], 2020. p. 50–55.

SHANNON, C. E. Communication in the presence of noise. **Proceedings of the IRE**, IEEE, v. 37, n. 1, p. 10–21, 1949.

SRIVASTAVA, N. et al. Dropout: a simple way to prevent neural networks from overfitting. **The journal of machine learning research**, JMLR. org, v. 15, n. 1, p. 1929–1958, 2014.

STEVENS, S. S.; VOLKMANN, J.; NEWMAN, E. B. A scale for the measurement of the psychological magnitude pitch. **Journal of the Acoustical Society of America**, v. 8, n. 3, p. 185–190, 1937.

Vigilancia Entomologica e Controle Vetorial do Ministerio da Saude. **Diretrizes Nacionais para Prevencao e Controle das Arboviroses Urbanas: Vigilancia Entomologica e Controle Vetorial OPAS/OMS 2025**. 2025. Available from Internet: <<https://www.gov.br/saude/pt-br/centrais-de-conteudo/publicacoes/svsa/dengue/diretrizes-nacionais-para-prevencao-e-controle-das-arboviroses-urbanas-vigilancia-entomologica-e-controle-vetorial/pdf/view>>.

WARDEN, P.; SITUNAYAKE, D. **TinyML Machine Learning with TensorFlow Lite on Arduino and Ultra-low-power Microcontrollers**. O'Reilly, 2020. ISBN 9781492052043. Available from Internet: <<https://books.google.com.br/books?id=sB3mxQEACAAJ>>.